队伍编号	MCB2302397
赛道	B

基于特征融合与集成学习的电商供应链分层预测方法

摘 要

需求预测往往是电商平台智能供应链的决策基础，它可以让管理者提前预知各地的需求，从而将库存提前放在靠近需求的仓库中，显著地降低库存成本，同时保证商品的按时履约。本文对各商品的历史出货量数据进行挖掘与分析，构建了基于数理特征与专家经验的多模型集成预测框架，旨在协助电商平台优化库存管理，更好地规划和定制促销活动，提高库存周转率。

针对问题一，需要对商品序列进行分类并预测未来需求量。本文首先根据附件数据构建全体需求量序列集。考虑部分字段信息覆盖了功能场景、仓库区域等组成的先验分类知识，先基于 8 个字段特征进行第一层级分类。然后，采用 K-means 算法对第一层级类别进行细化聚类，建立 9 个不同尺度的分类标准。针对预测问题，本文提出了一种基于自顶向下和分层聚类的集成预测方案。首先，基于 9 个分类标准获得各类别下总的需求量序列。然后，基于特征工程构建各类别汇总序列的特征集，分别采用指数平滑、Auto ARIMA、XGBoost、LightGBM、LSTM 等 5 个算法对各分类标准各类别下的总需求量序列构建预测模型，并设计合适的指标评估各算法的优劣。最终，采用 9 个分类标准下的模型预测值的集成量作为最后的预测输出，结果表明融合后的模型获得更高的 1-WMAPE 指标且预测更为稳定。

针对问题二，需要预测新出现序列组合的未来需求量。首先，将附件 1 与附件 5 的数据整合（2206 条），对所有组合截取等维长度的序列并提取数理特征，并将构建的数理特征与独热编码后的 8 个解释变量特征汇整，采用 K-means 算法对所有实例进行聚类（2206 条序列聚为 14 类）。然后，设置滑动窗口对聚类后的各类别序列进行特征提取和样本集构建，采用 SVM、XGBoost、LightGBM、LSTM、KNN 算法分别训练预测模型，基于 5 折交叉验证法调整模型参数并评估算法性能。在预测阶段，首先对待预测实例进行分类，采用类别下的 5 个模型预测值的集成量作为最后的预测输出，结果表明集成后预测模型可以挖掘出更为显著的相似序列特征，实现对新组合的精准预测。

针对问题三，需要预测规律性大型促销活动期间各商品的未来需求量。首先，提取附件 1 中各商品每月 1 日-11 日的出货量数据（共 5 个月）组成日常序列，并将各商品以月为基准取均值得到日常均值序列。提取附件 6 中各商品 1 日-11 日的出货量数据组成促销序列。然后，根据日常均值数据与促销数据计算各商品的促销力度值，基于问题一提出的集成预测框架对各商品在 6 月 1 日-6 月 20 日的日常值进行预测。最后，将预测结果与对应的促销力度值求积，得到最终的促销预测值，结果表明构建的事件影响模型可以分析出促销活动期间商品序列的波动规律，使预测值更符合促销期间的实际情况。

关键词：特征融合；集成学习；分层时序预测；LightGBM；K-means；LSTM

目　录

1 问题重述 2
1.1 问题背景 2
1.2 问题提出 2
2 问题分析 2
2.1 问题一的分析 2
2.2 问题二的分析 4
2.3 问题三的分析 5
3 模型假设 6
4 符号说明 6
5 问题一模型建立与求解 6
5.1 基于数理特征与专家经验的分层时序聚类算法 7
5.1.1 聚类特征提取 7
5.1.2 K-means 聚类算法构建 9
5.1.3 聚类结果评估 9
5.2 基于自顶向下和分层聚类的集成预测算法 10
5.2.1 指数平滑预测模型构建 11
5.2.2 Auto ARIMA 预测模型构建 12
5.2.3 XGBoost 预测模型构建 13
5.2.4 LightGBM 预测模型构建 13
5.2.5 LSTM 预测模型构建 14
5.3 任务预测与性能评价 14
6 问题二模型建立与求解 19
6.1 基于融合特征的单层聚类算法 19
6.1.1 特征提取 19
6.1.2 K-means 聚类算法构建 20
6.1.3 聚类结果评估 21
6.2 多模型集成预测算法 22
6.2.1 预测特征提取 23
6.2.2 预测算法构建 23
6.3 任务预测与性能评价 24
7 问题三模型建立与求解 25
7.1 问题分析与模型建立 25
7.2 预测结果分析 27
8 模型评价与推广 27
8.1 模型的优点 27
8.2 模型的缺点 28
8.3 模型的推广 28
参考文献 28
附录 29

1 问题重述

1.1 问题背景

电商平台存在着上千个商家，他们会将商品货物放在电商配套的仓库，电商平台会对这些货物进行统一管理。通过科学的管理手段和智能决策，大数据智能驱动的供应链可以显著降低库存成本，同时保证商品的按时履约。需求预测往往是智能供应链的决策基础，它可以让管理者提前预知各地的需求，从而将库存提前放在靠近需求的仓库中，此时的预测任务为：根据历史一段时间的需求量，预测各仓库中各商品未来需求，“预测维度”即为不同商家在各仓库中存放的各种商品每天的数量。一般来说，企业会首先根据数据的历史情况，分析出需求量序列的数理特征，对相似的需求量序列进行归类，并根据分类结果做到更加精准的预测。

然而，在实际的电商供应链预测任务中，常常会出现多种不同的问题。例如，部分商品的销售时间过短、仓库存在新增或切换等情况，导致该预测维度下历史数据过少；另外，部分大型促销期间货量的陡增并由此带来的不规律性，也给需求量的精准预测带来了不小的难度。此时便需要通过算法得到历史一般规律，找出相似的历史情况（如相似的仓或商品），从而实现精准预测。

1.2 问题提出

基于前述研究背景，该题目给出了 6 个信息附件和 3 个答案附件。基于 6 个信息附件内容，拟要解决的研究问题如下：

问题 1：使用附件 1-4 中的数据，预测出各商家在各仓库的商品 2023-05-16 至 2023-05-30 的需求量，并将预测结果填写在结果表 1 并上传至竞赛平台，并对模型的预测性能进行评价。另外讨论：根据数据分析及建模过程，这些由商家、仓库、商品形成的时间序列如何分类，使同一类别在需求上的特征最为相似。

问题 2：当出现新的商家+仓库+商品维度（附件 5），讨论这些新出现的预测维度如何通过历史附件 1 中的数据进行参考，找到相似序列并完成这些维度在 2023-05-16 至 2023-05-30 的预测值。把预测结果填写在结果表 2，并上传至竞赛平台。

问题 3：每年 6 月会出现规律性的大型促销，为需求量的精准预测以及履约带来了很大的挑战。附件 6 给出了附件 1 对应的商家+仓库+商品维度在去年双十一期间的需求量数据，参考这些数据，给出 2023-06-01 至 2023-06-20 的预测值。把预测结果填写在结果表 3，并上传至竞赛平台。

2 问题分析

2.1 问题一的分析

问题一主要是基于附件 1-4 中的数据，对各商家在各仓库的商品未来 15 天的需求量进行预测，并分析由商家、仓库和商品形成的需求量序列的数理特征，对相似的需求量序列进行归类，根据分类结果提高需求量预测的精度。因此，问题一主要涉及聚类和

预测两个任务。针对上述任务，对附件 1-4 进行分析，梳理的经验特征有 8 个，包括字段名：商品一级分类、二级分类等。这些字段信息既包含了各需求量序列本身的特征属性，也覆盖了一些由行业标准、功能场景、仓库区域等组成的先验分类知识。如字段“qty_seq”为附件中所有商家-仓库-商品的组合，由这些组合构成的数据集无显式的内在关联，只能依赖聚类算法抽象处理；而字段“category1”-“warehouse_region”可以利用先验知识进行数据的初步划分，再借助聚类算法进行二次规整，使得划分的合理性和解释性更强。比如，往往规模大的商家出货量较多，对应的需求量序列变化趋势相似。因此，在进行序列分类时，可以先依据该字段名和特征类别对所有实例进行初步划分，而后采用聚类算法对相应类别进行细化。

具体地，针对分类问题，本文提出了一种基于数理特征与专家经验的分层时序聚类算法，从需求量序列的时域分布、稳定性、所属的商家、商品、仓库类别划分等尺度提取特征构建分类标准，算法架构如图 2.1 所示：

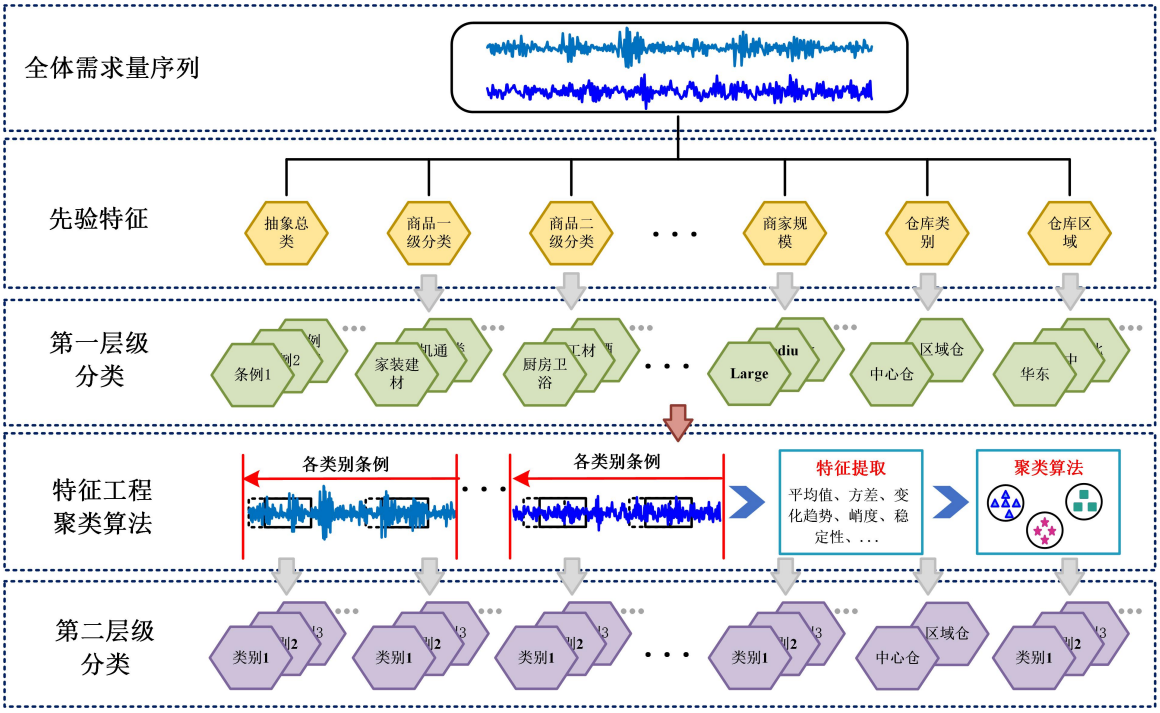


图 2.1 基于数理特征与专家经验的分层时序聚类算法

首先，基于附件 1-4 的数据构建全体需求量序列集。然后，根据表 2.1 中的经验特征“category1”-“warehouse_region”分别进行第一层级分类，获得 9 个分类标准，各分类标准下的类别数不等，如“category2”标准下有 79 类，“category3”标准下有 291 类，各类别所包含的实例数也不同。为了缩小各分类标准所含类别的个数，对第一层级分类结果中各类别数据进行数理特征提取[1-2]。最后，采用 Kmeans 聚类法[3]对每个分类标准下的类别数进行聚类，其目的在于将时序变化趋势差异较大的类别分离开，进而得到各分类标准下的最终类别数。比如，“category2”分类标准在第一层级下将数据按照市场经验划分为了 79 类，通过聚类算法在第二层级下被抽象分为 4 类。聚类后各标准对应的类别数如表 2.1 所示。

表 2.1 聚类后的类别数

分类标准	第一层级类别数	第二层级类别数
qty_seq	1996	5
category1	23	3
category2	79	4
category3	291	4
seller_category	17	4
inventory_category	4	4
seller_level	4	4
warehouse_categoty	2	2
warehouse_region	7	3

针对预测问题，本文提出了一种基于自顶向下和分层聚类的集成预测算法[4-5]。首先，对 9 个分类标准各类别下包含的实例进行求和处理，获得各类别下总的需求量序列，共计 33 条。然后，对各类别下总的需求量序列进行特征提取，获得表征各类别需求量序列变化趋势的显著特征。最后，分别采用极致梯度提升（XGBoost）[6]、轻量级梯度提升机器学习（LightGBM）[7]、长短时记忆网络（LSTM）[8]、随机森林[9]等预测算法对各类别下的总需求量序列构建时序预测模型（各算法下均构建 33 个模型），并设计合适的指标评估各算法的优劣，进而选择最优的预测算法。在预测阶段，针对待预测的需求量序列，先按照 9 个分类标准将其分别划分到第二层级的各类别中，并基于各类别构建的模型采用自顶向下的方案得到 9 个预测值。最终，采用 9 个模型预测值的集成量作为最后的预测输出，具体的实现流程如图 2.2 所示：

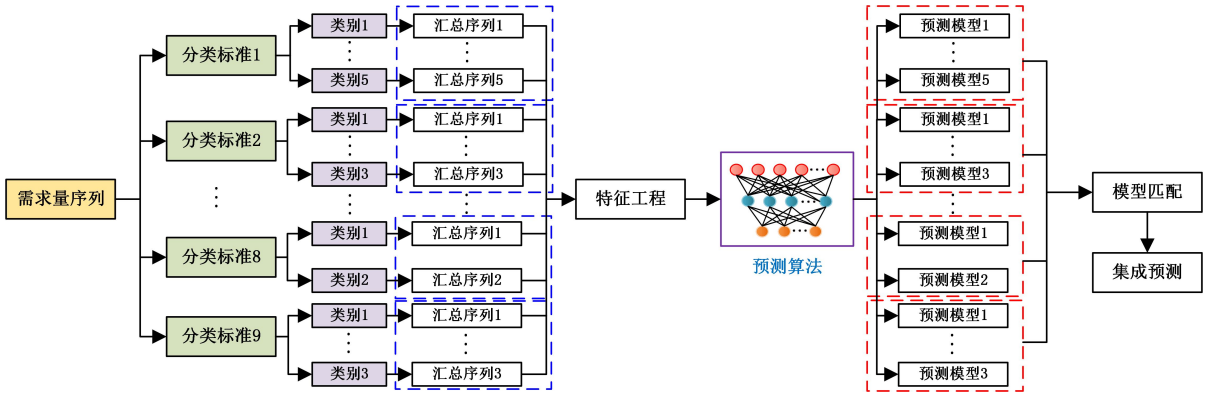


图 2.2 基于分层聚类的集成预测算法实施架构

2.2 问题二的分析

问题二主要是考虑出现了新的商家+仓库+商品维度，对这些实例未来 15 天的需求量进行预测。这些实例主要有以下特征：1）出货量序列信息不一致，部分实例由于是新上市或改变了原先存放的仓库，导致不同序列长度不一致；2）出现了新的第一层级类别，使得难以直接借助问题一的架构进行分类和预测。

针对上述问题，首先，将附件 1 与附件 5 的数据整合（1996 条+210 条），发现整合后的商家-仓库-商品序列在 2023/4/23-2023/5/15 期间需求量序列维度一致。对

2023/4/23-2023/5/15 期间所有数据提取数理特征，并与独热编码后的 8 个解释变量特征（商品一级类别等）汇整，构建特征集合，采用 K-means 算法对所有实例进行聚类，将 2206 条序列分为 14 类。然后，采用滑动窗口对聚类后各类别序列进行特征提取，采用支持向量机（SVM）、XGBoost、LightGBM、LSTM 等算法分别训练预测模型。在预测阶段，首先对待预测实例进行分类，基于该类别中 n 个算法模型得到 n 个不同的预测值，并将各模型预测值进行集成得到最终的预测输出，整体流程如图 2.3 所示。

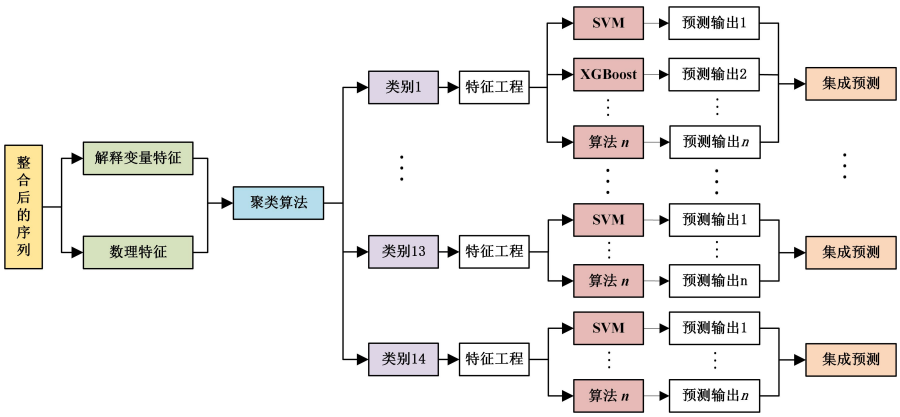


图 2.3 问题二求解流程

2.3 问题三的分析

问题三主要是考虑在出现规律性大型促销活动时，对实例在 2023/06/01-2023/06/20 期间的需求量进行预测。根据实际分析，将 2023/06/01-2023/06/20 期间活动峰值确定在 2023/06/18（京东 618）。大型促销活动期间，商品的需求量将出现以下特点[10]：1）序列无序性增强：在促销活动期间，商品出货量通常会经历销售峰值，出货量显著增加，商品库存快速减少，导致序列的规律性大幅降低；2）样本稀少：规律性的促销活动只在一段时间内定期出现，相应的历史数据较为稀少，样本的数理特征难以挖掘和关联。

针对上述问题，首先，提取附件 1 中各商品每月 1 日-11 日的出货量数据（共 5 个月）组成日常序列，并将各商品以月为基准取均值得到日常均值序列。提取附件 6 中各商品 1 日-11 日的出货量数据组成促销序列。然后，根据日常均值数据与促销数据计算各商品 1 日-11 日的促销力度值。基于问题一提出的集成预测框架对各商品 6 月 1 日-20 日的日常值进行预测。最后，将预测结果与对应的促销力度值求积，得到最终的促销预测值，整体流程如图 2.4 所示。

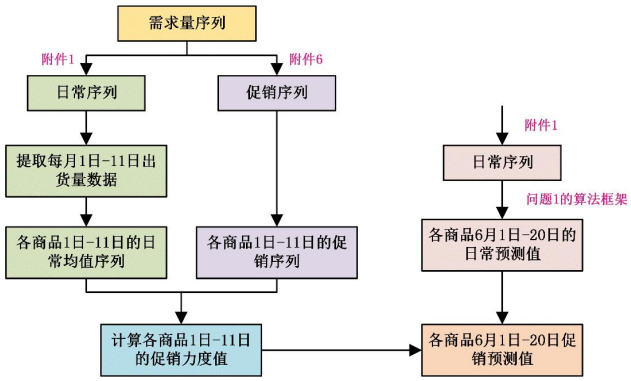


图 2.4 问题三求解流程

3 模型假设

1、假设所调查的商品需求量序列的数据都比较准确；
2、假设附件中的数据在一定时间内是稳定的，除季节性、趋势性和周期性特征外，统计特性在相近时间段内基本保持一致；
3、假设附件中数据点之间存在一定程度的自相关性和依赖性，即过去的数据点对未来的数据点有影响；
4、同一由商家-仓库-商品组成的序列在规律性促销期间出货流量无显著性差异；

4 符号说明

符号	符号说明
C_i	第 i 个分类标准
$\bar{x}$	序列 x 的均值
σ	标准差
$Kurtosis$	峰度
S	偏度
$\hat{y}$	序列的预测值
α	平滑系数
d	滑动窗口长度
W	权重系数
δ	K-means 收敛阈值
f	傅里叶频率
$I(\bullet)$	指示函数

5 问题一模型建立与求解

问题一主要涉及分类和预测两个任务。针对聚类任务，本文提出了一种基于数理特征与专家经验的分层时序聚类算法。首先，基于附件 1-4 的数据构建全体需求量序列集。然后，根据表 1 中的经验特征分别进行第一层级分类。为了缩小各分类标准所含类别的个数，对第一层级分类结果中各类别数据进行数理特征提取。最后，采用 K-means 对每个分类标准下的类别数进行聚类，获得各标准下的第二层级细化类别。

针对预测问题，本文提出了一种基于自顶向下和分层聚类的集成预测算法。首先，对 9 个分类标准各类别下的实例进行汇总，获得各类别下总的需求量序列。然后，对各类别下总的需求量序列进行特征提取，获得表征各类别需求量序列变化趋势的显著特征。最后，分别采用极致梯度提升（XGBoost）、轻量级梯度提升机器学习（LightGBM）、长短时记忆网络（LSTM）等算法对各分类标准类别下的总需求量序列构建预测模型，并设计合适的指标评估各算法的优劣。在预测阶段，基于各分类标准构建的模型采用自顶向下的方案得到 9 个预测值并将其集成作为最终的预测量。关键流程如图 5.1 所示。

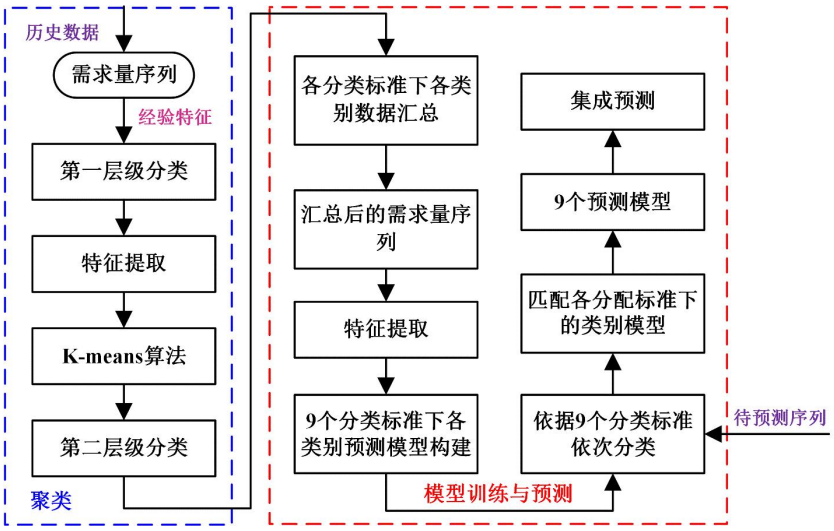


图 5.1 问题一模型建立与求解流程

5.1 基于数理特征与专家经验的分层时序聚类算法

5.1.1 聚类特征提取

根据附件 1 中的数据，首先构建由商家-商品-仓库形成的需求量序列，共计 1996 条时间序列实例，其中每个实例是由 2022/12/01-2023/5/15 的出货量组成，样本长度为 166。附件 2-4 中包含了 8 个经验特征：商品一级分类、商品二级分类、商品三级分类、商家分类、库存分类、商家规模、仓库类别、仓库区域。分别统计 8 个经验特征的类别数，结果如表 2.1 所示。这些经验特征覆盖了一些由功能场景、仓库区域等组成的先验分类知识，可以为需求量序列的划分提供经验知识。然而，若直接以经验特征划分类别存在两个问题：1）部分经验特征包含的类别数较多，如指标“商品二级分类”包含厨房卫浴、电工电料、洋酒等 79 个类别，类别繁杂且难以直接合并；2）若仅依靠 8 个专家指标将忽视需求量序列的数理特征和本身属性，难以保证各类别下序列的时序趋势相似。

针对上述问题，构建分层聚类算法，融合抽象特征与领域知识，为不同序列的划分提供多个尺度的分类依据。其中，第一层级为各经验特征形成的分类标准，对于第一层级分类标准下类别数较多的特征采用聚类算法进行二次分类，待聚类特征如表 5.1 所示。

表 5.1 经验特征与聚类标准

特征字段名	编号	第一层级类别数	是否聚类
qty_seq	分类标准 1（C_1）	1996	✓
category1	分类标准 2（C_2）	23	✓
category2	分类标准 3（C_3）	79	✓
category3	分类标准 4（C_4）	291	✓
seller_category	分类标准 5（C_5）	17	✓
inventory_category	分类标准 6（C_6）	4	×
seller_level	分类标准 7（C_7）	4	×
warehouse_categoty	分类标准 8（C_8）	2	×
warehouse_region	分类标准 9（C_9）	7	✓

其中，分类标准 1 表示每个需求量序列为一个类别，为最细化的情形。从表 5.1 可以看出，共有 6 个分类标准需要进行二次聚类。其中，每个由商家-商品-仓库组成的需求量序列长度为 166，包含了 5 个半月的出货量数据。为了捕获需求量序列的周期性、季节性等特征，同时考虑实际商品销售时，相近日期出货量的强相关性，对序列进行划分，以半个月为周期将原始的需求量序列分为 11 段长度为 15 的新序列。针对每段新序列，构建如下的聚类特征：**平均值、标准差、最大值、最小值、趋势性强度、峰度和偏度**，共计 7 个特征。其中，趋势性强度、峰度和偏度的定义与计算方式如下：

1）趋势性强度：衡量一段时间序列数据中趋势的强度或趋势的持续性。具体地，采用线性回归拟合时间序列数据，得到如下方程：

$$y = k_a x + b \tag{5-1}$$

其中，y 是需求量序列，x 是时间，k_a 为斜率反映序列趋势的方向和强度。正斜率表示上升趋势，负斜率表示下降趋势，斜率的绝对值越大，趋势的强度越大。

2）峰度：峰度用于度量随机变量概率分布的陡峭程度，表征了数据分布的尖锐程度，或者说是数据集中在均值附近的程度。计算公式如下：

$$Kurtosis = \frac{1}{n}\sum_{i=1}^{n}\left[\left(\frac{x_i - \bar{x}}{\sigma}\right)^4\right] \tag{5-2}$$

其中，n 为序列数据点的长度，x_i 为每个数据点，$\bar{x}$ 为均值，σ 为标准差。

3）偏度：偏度用来度量随机变量概率分布的不对称性。用来衡量数据分布的偏斜程度，即数据分布的尾部在均值的哪一侧更加重或更加轻。计算公式如下：

$$S = \frac{1}{n}\sum_{i=1}^{n}\left[\left(\frac{x_i - \bar{x}}{\sigma}\right)^3\right] \tag{5-3}$$

各分段序列长度为 15，由上述 7 个特征表示，则每条序列可以由 77 个特征点进行刻画。对于分类标准 1，需要对 1996 条序列直接进行抽象聚类，构造的聚类特征集为 1996*77。对于分类标准 2-5 和 9，由于在第一层级已经借助先验知识进行了数据的初步划分，则聚类的目标是将第一层级的类别进行二次细化聚类，使得类别划分的合理性更强，具体的聚类流程如图 5.2 所示。

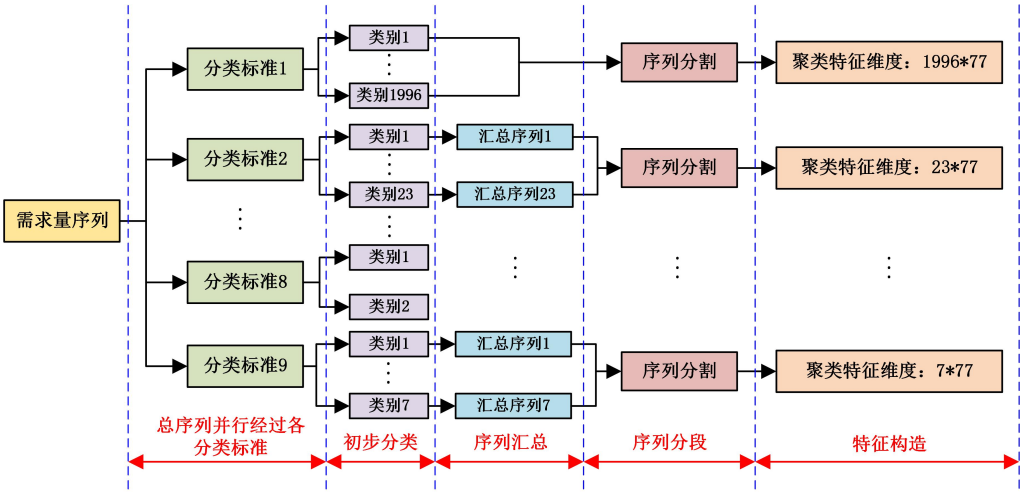


图 5.2 各分类标准特征提取流程

5.1.2 K-means 聚类算法构建

基于上述构造的特征集对各分类标准的第一层级类别进行规整，采用 K-means 算法进行聚类细化，算法执行流程如下：

1）设定 k 值范围 $k \in [1, k_{\max}]$，计算每个数据到 k 个初始化聚类中心的距离，将数据分到距离聚类中心最近的数据集中，当所有数据对象都划分以后，就形成了 k 个数据集。

2）重新计算每个簇的数据对象的均值，将均值作为新的聚类中心。

3）计算每个数据对象到新的 k 个初始化聚类中心的距离，重新划分，直到所有的数据对象无法更新到其他的数据集中。

4）采用肘部法则和轮廓系数评估聚类结果，进而选择合适的 k 值。

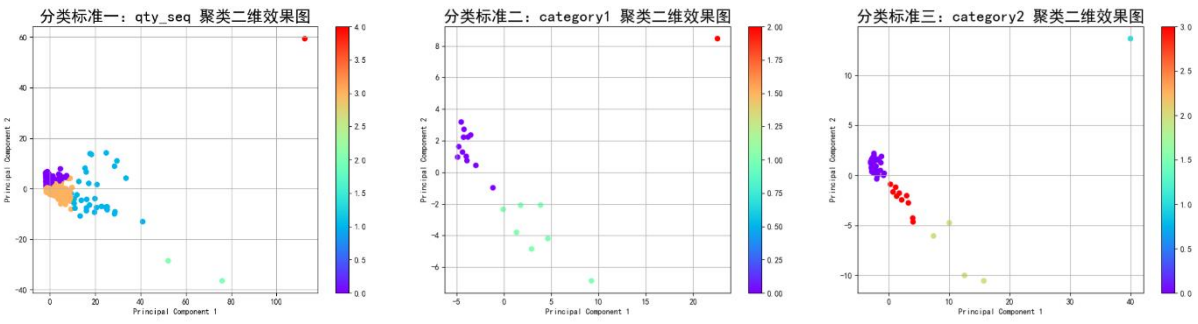
具体地，针对每个分类标准的特征集进行 K-means 聚类，得到的聚类结果如表 5.2 所示。其中，第二层级类别数相对于第一层级显著减少，使得类别划分更为详细且同一类别序列的特征分布更为相似。本文提供了 9 个分类标准，当出现新的序列时，可以选择任意分类标准对其进行分类。

表 5.2 K-means 聚类后的类别数

特征字段名	第一层级类别数	第二层级类别数
qty_seq	1996	5
category1	23	3
category2	79	4
category3	291	4
seller_category	17	4
inventory_category	4	4
seller_level	4	4
warehouse_categoty	2	2
warehouse_region	7	3

5.1.3 聚类结果评估

基于 K-means 聚类方法，采用 5.1.1 中所述的几类特征指标对第一层级类别数较多的特征进行聚合，其中分类标准 2-5 和 9 需要进行 K-means 聚类细化。将聚类后的结果使用主成分分析（PCA）进行降维并可视化聚类结果，结果如图 5.3 所示。



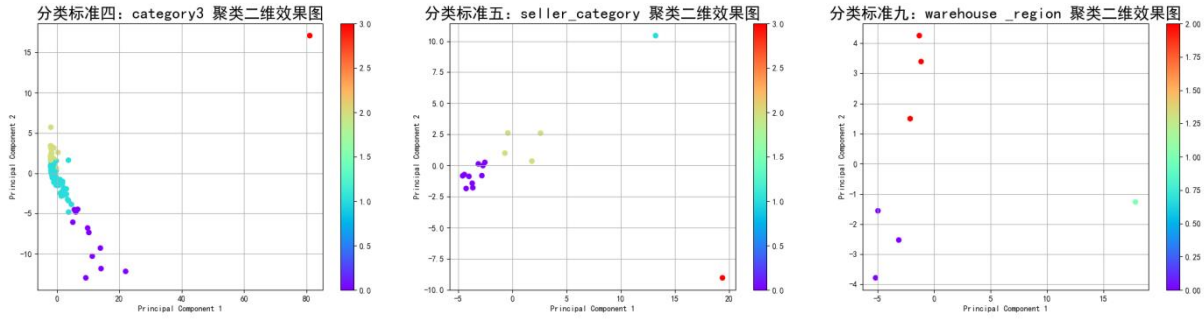


图 5.3 K-means 聚类后的可视化分类图

为了更直观的分析聚类效果，将聚类之后各分类标准下属于同一类别的需求量序列绘制为折线图，结果如图 5.4 所示。从图中可以看出，本文构建的聚类方案可以较好地挖掘出序列中类分离的显著特征。基于第一层级的字段名分类利用到了领域的先验知识，使得类别划分的可解释性增强。提出的 9 个分类标准都能较好地时间序列进行分类，使同一类别在需求上的特征最为相似。

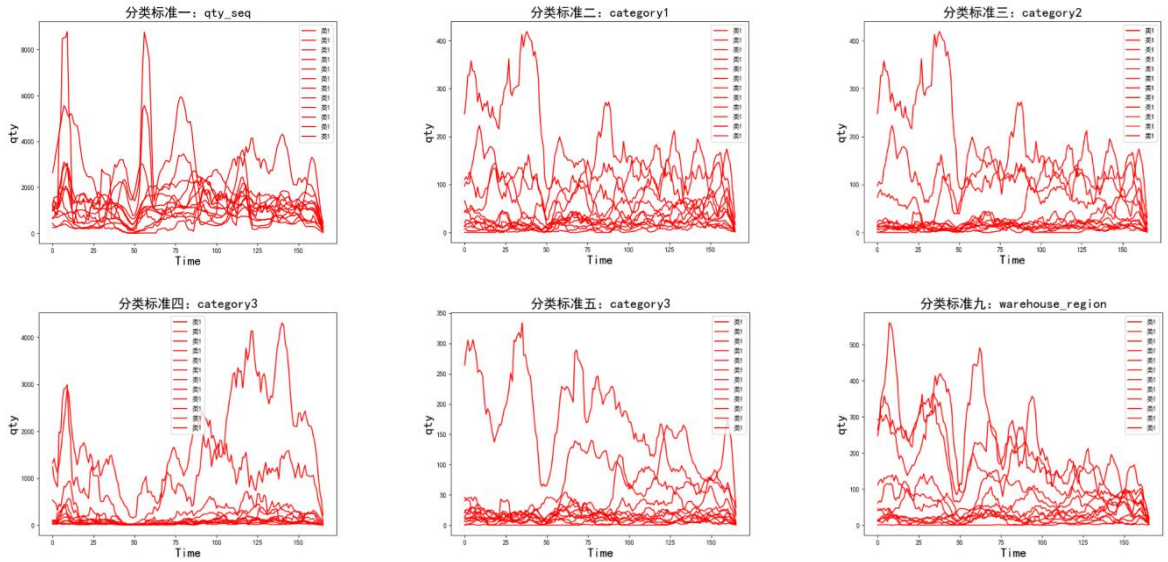


图 5.4 各分类标准下类别 1 中序列的分布情况

5.2 基于自顶向下和分层聚类的集成预测算法

针对预测任务，本文提出了一种基于自顶向下和分层聚类的集成预测算法。首先，对 9 个分类方案各类别下包含的实例进行求和处理，获得各类别下汇总的需求量序列，共计 33 条。然后，对各分类方案的各类别下汇总的需求量序列进行特征提取，获得表征各类别需求量序列变化趋势的显著特征。最后，分别采用指数平滑（ES）、自调节的差分整合移动平均自回归模型（AutoARIMA）、XGBoost、LightGBM、LSTM 预测算法对各类别下的总需求量序列构建时序预测模型（各算法下均构建 33 个模型），并设计合适的指标评估各算法的优劣，进而选择最优的预测算法。

在预测阶段，针对待预测的需求量序列，先按照 9 个分类标准将其分别划分到第二层级的各类别中，获得各类别汇总的需求量序列。然后，将单序列的预测任务转化为其所属类别汇总需求量的预测任务，并基于各类别构建的时序预测模型获得汇总序列的预

测值。其次，在各分类标准下，计算待预测序列占其归属类别汇总的序列的比例，采用自顶向下的方案获得单序列的预测值。最后，将 9 个分类标准下的预测值进行集成作为最后的预测输出，框架执行流程如图 5.5 所示。

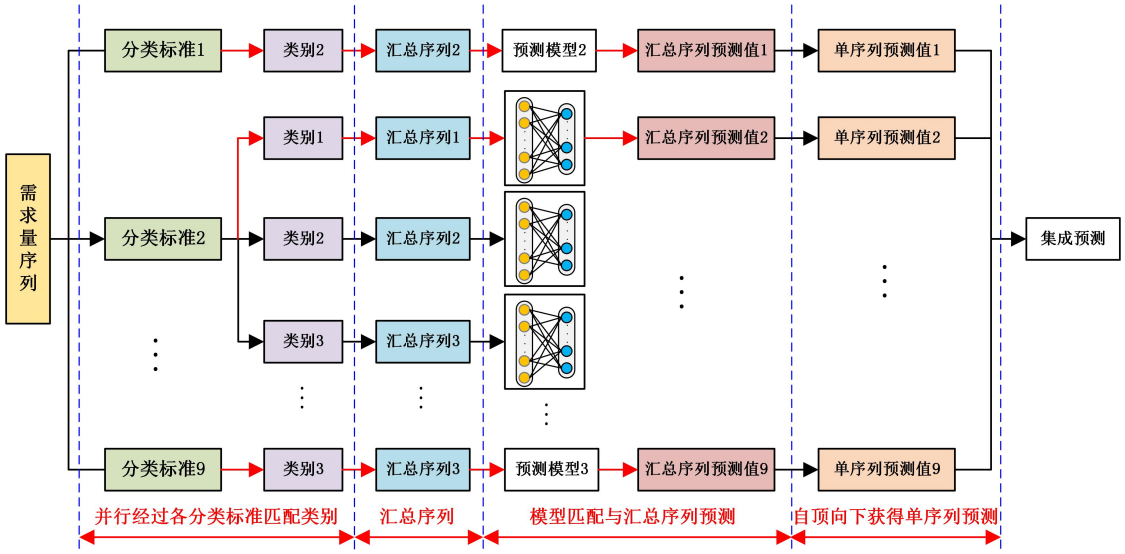


图 5.5 基于自顶向下和分层聚类的预测任务流程

注意到，在训练阶段，9 个分类标准下的 33 个预测模型均基于各类别汇总后的需求量序列进行构建，因此在对单需求量序列进行预测时，需要先匹配其在 9 个分类标准下的类别归属，将任务转换为汇总序列的预测，然后计算单需求量序列在其所属类别汇总需求量序列的比例因子，采用自顶向下的方案求得单序列的预测值：对层次结构的顶层进行预测，然后通过预定义的比例将其分解到所属类别的节点。

假设 y_A 为某条需求量序列，长度为 166，现需对其未来 15 天的需求量进行预测。在分类标准 1 中其被划分到了类别 2，类别 2 中汇总后的需求量序列为 $y_{C_{12}}$，为类别中所有实例的加和，长度也为 166。采用预测模型对汇总后的序列 $y_{C_{12}}$ 进行预测，得到未来 15 天汇总序列的预测值 $\hat{y}_h$，则单需求量序列的预测值可以通过下式计算：

$$\hat{y}_{A,h} = p \bullet \hat{y}_h, \quad p = \frac{1}{T}\sum_{t=1}^{T}\frac{y_{A,t}}{y_{C_{12},t}} \tag{5-4}$$

式中，p 为比例因子，T 表示天数。比例因子通过计算 T 天内单序列每天的出货量占汇总序列每天出货量的均值求得。在本文中，采用预测任务之前的 30 天每天的出货量占加和序列每天出货量比重的均值进行表征。依次计算待预测序列在各分类标准下的预测值，采用 9 个预测量的均值作为最后的预测量：

$$\hat{y}_A = \frac{1}{9}\sum_{h=1}^{9}\hat{y}_{A,h} \tag{5-5}$$

基于图 5.5 的框架，分别采用 ES、Auto ARIMA、XGBoost、LightGBM、LSTM 算法训练预测模型，并设计合适的指标评估各算法的优劣，进而选择最优的预测算法。

5.2.1 指数平滑预测模型构建

指数平滑又称为指数修匀，其原理是将历史数据进行加权平均作为未来时刻的预测

结果，并且随着过去观测值离预测值距离的增大，权重呈指数型衰减。具体的算法流程如下：

1）对原始数据进行预处理，消除时间序列中偶然性的变化，提高收集的数据中近期数据在预测中的重要程度。

2）考虑趋势与季节性的影响，构建三次指数平滑模型：

$$S_{t+1}^{(1)}=\alpha X_t+(1-\alpha)S_t^{(1)},S_{t+1}^{(2)}=\alpha S_t^{(1)}+(1-\alpha)S_t^{(2)},S_{t+1}^{(3)}=\alpha S_t^{(2)}+(1-\alpha)S_t^{(3)} \quad (5\text{-}6)$$

式中，S_t 为第 t 天的预测值，X_t 为第 t 天的实际值，α 为平滑系数。预测未来 T 天：

$$y_{t+T}=A_t+B_tT+C_tT^2 \quad (5\text{-}7)$$

式中，

$$\begin{cases} A_t=3S_t^{(1)}-3S_t^{(2)}+S_t^{(3)},\ C_t=\left(\dfrac{\alpha}{2(1-\alpha)^2}\right)\left(S_t^{(1)}-2S_t^{(2)}+S_t^{(3)}\right) \\ B_t=\left(\dfrac{\alpha}{2(1-\alpha)^2}\right)\left((6-5\alpha)S_t^{(1)}-2(5-4\alpha)S_t^{(2)}+(4-3\alpha)S_t^{(3)}\right) \end{cases} \quad (5\text{-}8)$$

3）使用历史时间序列数据训练指数平滑模型，通过趋势调整、添加趋势修正值改进指数平滑法的预测结果。最后，使用训练好的模型生成未来时间点的预测值。

5.2.2 Auto ARIMA 预测模型构建

ARIMA 模型是一种广泛应用的时间序列预测模型，它结合了自回归（AR）和移动平均（MA）的概念，具有较好的灵活性和准确性。Auto ARIMA 提供了模型阶次自适应寻优方案，指定一个阶数上限和信息准则计算方法，从而找到信息准则最小的阶数组合，避免了传统 ARIMA 复杂的参数调整过程。其中，ARIMA 模型的公式为：

$$Y_t=c+\varphi_1Y_{t-1}+\varphi_2Y_{t-2}+\ldots+\varphi_pY_{t-p}+\theta_1\varepsilon_{t-1}+\theta_2\varepsilon_{t-2}+\ldots+\theta_q\varepsilon_{t-q}+\varepsilon_t \quad (5\text{-}9)$$

式中，Y_t 为考虑的时间序列数据，φ_1 到 φ_p 为 AR 模型的参数，θ_1 到 θ_q 为 MA 模型的参数。ε_t 为 t 时间点上的误差项，c 为常数项。对于第二层级各类别汇总的需求量序列分别训练 Auto ARIMA 模型（共计 33 个），算法构建过程如图 5.6 所示。

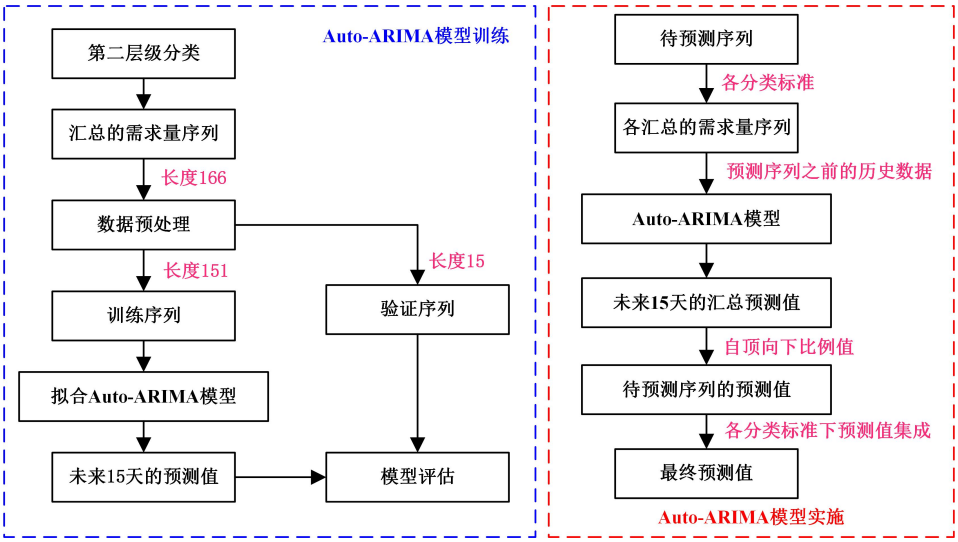


图 5.6 Auto ARIMA 模型的构建流程

其中，训练样本为 2022/12/01-2023/4/30 期间出货量组成的长度为 151 的汇总序列。验证样本为汇总实例未来 15 天的出货量，即 2023/5/01-2023/5/15 期间的出货量。在模型实施阶段，用各需求量序列 2023/5/15 之前的历史数据经过构建好的 Auto ARIMA 模型预测 2023/05/16-2023/05/30 的需求量。

5.2.3 XGBoost 预测模型构建

XGBoost 模型是梯度分类和回归问题的有效实现，它是决策树算法的集合，其中新树修复了那些已经属于模型的树的错误，将添加树直到无法对模型进一步改进为止。XGBoost 提供了随机梯度提升算法的高效实现，并提供了一组模型超参数，这些参数旨在提供对模型训练过程的控制。已知训练数据集 $\{(x_1,y_1),(x_2,y_2),...,(x_n,y_n)\}$，损失函数 $l(y_i,\hat{y}_i)$，正则化项 $\Omega(f_k)$，则整体目标函数为：

$$L(\phi)=\sum_i l(y_i,\hat{y}_i)+\sum_k \Omega(f_k) \tag{5-10}$$

式中，$L(\phi)$ 是线性空间的表达，i 为样本编号，k 表示数的编号，$\Omega(f_k)$ 反映了 k 棵树的复杂度。当 XGBoost 模型构建完成，第 i 个样本的预测值为：

$$\hat{y}_i=\sum_{k=1}^{K} f_k(x_i) \tag{5-11}$$

XGBoost 为有监督学习，针对问题一中的任务，采用 XGBoost 构建预测模型的核心是数据集的设计。取 2022/12/01-2023/4/30 期间出货量组成的长度为 151 的汇总序列为训练集，2022/5/01-2023/5/15 期间的出货量为验证集，设置滑动窗口为 15，截取长度为 15 的切割序列作为输入样本 x_i，切割序列后一天的需求量为输出标签 y_i，通过滑动窗口构建最终的训练集合，如图 5.7 所示。当模型训练结束后，按照图 5.5 的流程进行预测。

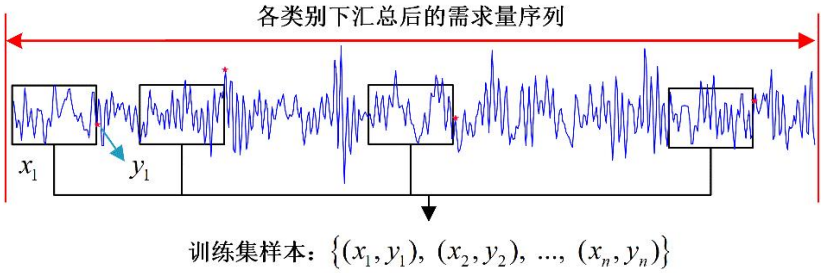


图 5.7 XGBoost 训练集构造

5.2.4 LightGBM 预测模型构建

LightGBM 是个快速、分布式、高性能的基于决策树算法的梯度提升框架。可以克服 XGBoost 空间消耗和计算量大的缺陷，并且能够在不损害准确率的条件下加快梯度提升决策树（GBDT）模型的训练速度，算法的基本流程如图 5.8 所示。

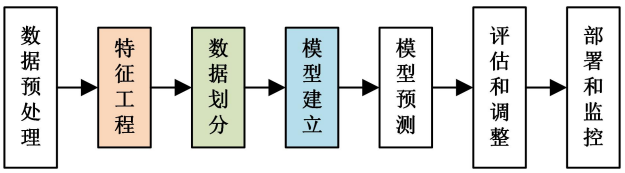


图 5.8 LightGBM 模型工作流程

与 XGBoost 相似，LightGBM 也为有监督学习，合理的特征工程会大幅提高模型的效果。因为树模型是基于分裂点的，它对于特征与特征之间的关系不敏感。为了捕获需求量序列的周期性、季节性、滞后性等特征，同时考虑在实际商品销售时，相邻日期出货量的强相关性，对于 2022/12/01-2023/4/30 期间出货量组成的长度为 151 的汇总序列进行分段，将其作为训练数据。设置滑动窗口为 30，截取长度为 30 的切割序列作为新数据，切割序列后一天的需求量为输出标签，获得的切割序列个数为：

$$N_s = l_s - d - 1 \tag{5-12}$$

式中，l_s 为原始序列的长度，d 为滑动窗口的长度。按照上述参数设定，切割后的序列个数为 120 个，长度为 30。针对每段新序列，构建如下的特征：**1）滞后特征**：包括各切割序列倒序 1 天到倒序 7 天的需求量之和；倒序 7 天、14 天、21 天、28 天的需求量均值和标准差；**2）数理特征**：切割序列的最大值和最小值；切割序列的趋势性强度；**3）周期性/季节性特征**：周期性/趋势性强度。共计 7 个特征，通过特征工程后，获得 120*7 的特征集，取 2022/5/01-2023/5/15 期间的出货量作为验证集训练 LightGBM 模型。当模型训练完成后，按照图 5.5 的流程预测 2023/05/16-2023/05/30 的需求量。

5.2.5 LSTM 预测模型构建

LSTM 是循环神经网络（RNN）的变体，它通过使用记忆单元和门控机制来解决传统 RNN 中的梯度消失和梯度爆炸问题。核心是由输入门、遗忘门、输出门和记忆单元组成。其中，输入门用于控制输入信息的更新，遗忘门用于控制记忆的遗忘，输出门用于控制输出信息的选择，记忆单元用于存储和更新信息。基于上述机制，能够挖掘出数据中的时序依赖关系，算法的基本流程如图 5.9 所示。

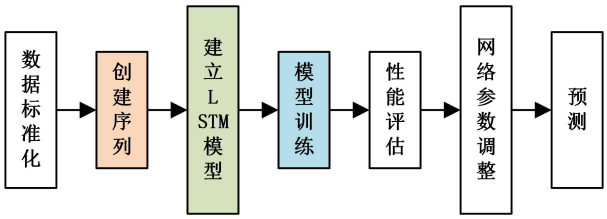


图 5.9 LightGBM 模型构建流程

LSTM 数据集的构造 XGBoost 相似。其中，训练样本为 2022/12/01-2023/4/30 期间各类别所包含实例出货量组成的长度为 151 的汇总序列，设置滑动窗口为 15，截取长度为 15 的切割序列作为输入样本，切割序列后一天的需求量为输出标签。验证样本为汇总实例未来 15 天的出货量，即 2023/5/01-2023/5/15 期间的出货量。

5.3 任务预测与性能评价

基于附件 1-4 中商品-商家-仓库在 2022/12/01-2023/5/15 期间形成的需求量序列和图 5.5 的框架，分别采用 ES、Auto ARIMA、XGBoost、LightGBM、LSTM 算法训练预测模型，各算法的拟合能力和在验证集上的表现如图 5.10 和 5.11 所示。从图中可以看出，提出的各算法均可以挖掘出时序数据中的抽象特征，实现对出货量的预测。其中，ES

和 Auto ARIMA 主要是基于统计学原理进行模型训练，预测输出较为平缓。XGBoost 和 LightGBM 基于决策树的架构进行模型训练，可以捕获序列中周期性、季节性、滞后性等特征，具有适中的计算复杂度。LSTM 主要是基于神经网络强大的学习能力来抽取序列中的抽象特征，在训练集上展现了优异的能力，较好地预测出未来需求量的走势。

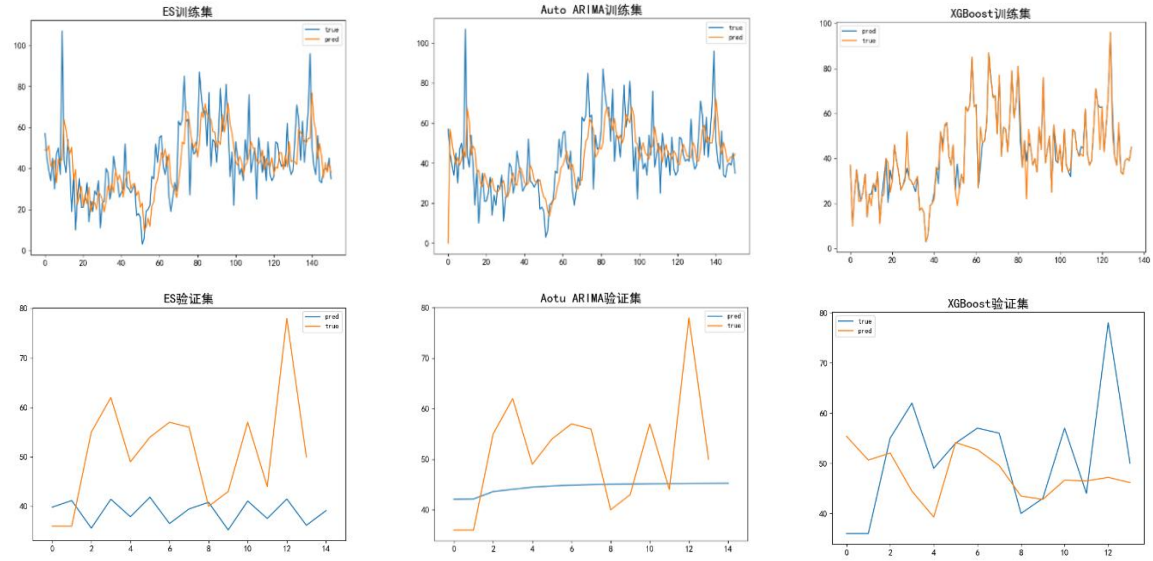


图 5.10 ES、Auto ARIMA 和 XGBoost 对部分类别的拟合与预测结果

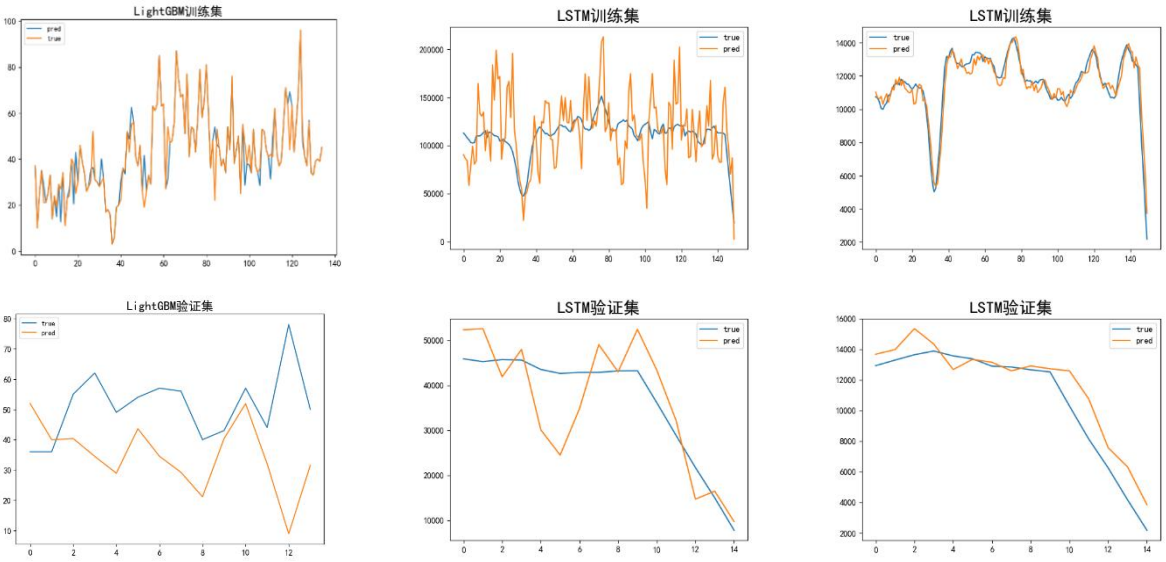


图 5.11 LightGBM 和 LSTM 对部分条例的拟合与预测结果

针对各类算法，均取 2023/05/01-2023/05/15 期间 15 天的出货量作为验证集。选取以下指标对各算法性能进行评价：

1）1-wmape：表示预测值与实际值之间的平均绝对百分比误差。考虑了不同预测值的权重以更好地反映不同数据点的重要性。较低的 WMAPE 值拥有更准确的预测，计算公式如下：

$$1-\text{wmape}=1-\frac{\sum|y_i-\hat{y}_i|}{\sum y_i} \tag{5-13}$$

式中，y_i 为验证集的真实值，$\hat{y}_i$ 为模型的预测值。由于部分序列在 2023/05/01-2023/05/15

期间真实值均为 0，为了避免分母奇异，采用修订后的下式进行计算：

$$1-\text{wmape}=1-\frac{\sum \text{len}(\hat{y}_i \neq 0)}{\sum \text{len}(y_i)} \tag{5-13}$$

式中，$\text{len}(\hat{y}_i \neq 0)$ 表示预测序列中值不为 0 的天数。修订后的指标反映了预测序列中不为 0 的个数占总预测长度的比例。

2）RMSSE：表示模型的均方根比例误差。它量化了模型的预测误差相对于一个简单基准模型（通常是历史平均值）的性能。它不受数据规模的影响，因此可以用于不同问题和数据集的比较。较低的 RMSSE 拥有更好的预测性能，其计算公式为：

$$\text{RMSSE}=\sqrt{\frac{\frac{1}{h}\sum_{t=n+1}^{n+h}(y_t-\hat{y}_t)^2}{\frac{1}{n-1}\sum_{t=2}^{n}(y_t-y_{t-1})^2}} \tag{5-14}$$

式中，n 为训练样本序列的长度，h 为验证集序列的长度。

3）MAPE：平均绝对百分比误差，用于度量预测值与实际观测值之间的百分比误差的平均值。有助于反映预测值相对于实际值的偏离程度，其不受数据的绝对值或单位的影响。较低的 RMSSE 拥有更好的预测性能，计算公式为：

$$\text{MAPE}=\frac{1}{h}\sum_{i=1}^{h}\left|\frac{y_i-\hat{y}_i}{y_i}\right| \tag{5-15}$$

4）RMSE：均方根误差，用于量化模型的预测值与实际观测值之间的差异。该指标较为稳定，不受异常值的干扰，不依赖于残差的分布，可解释性较强。较低的 RMSE 拥有更好的预测性能，计算公式为：

$$\text{RMSE}=\sqrt{\frac{1}{h}\sum_{i=1}^{h}(y_i-\hat{y}_i)^2} \tag{5-16}$$

基于上述指标，各算法预测性能的评价值如表 5.3 所示。从表中看出，在指标 1-WAMAPE 下，LightGBM 算法在各分类标准中均取得了较好的预测性能。虽然，Auto ARIMA 在 RMSSE、MAPE、RMSE 指标上与 LightGBM 具有相媲美的性能，但是它在验证集上的预测往往表现为输出一段平缓的均值，难以捕获序列中的滞后性和波动性特征。同时，针对实际预测任务，当待预测样本按照 9 个分类标准划分结束后，电商平台也可以依据 4 个指标选取各类别下最优的算法执行集成预测任务。

表 5.3 不同算法下预测性能评价值

平均评价指标		1-WAMAPE					RMSSE				
		LightGBM	ARIMA	EM	XGBoost	LSTM	LightGBM	ARIMA	EM	XGBoost	LSTM
数理特征	类别 1	0.7929	**0.8058**	0.7887	0.7451	0.7537	2.9965	2.3264	2.3310	3.5776	10.3783
	类别 2	**0.8967**	0.8733	0.8799	0.8568	0.7887	1.7860	1.3154	1.4094	2.5044	9.5255
	类别 3	0.7212	0.4428	0.5179	0.4178	**0.7225**	4.5650	7.3000	6.2816	9.5181	9.2683
	类别 4	**0.9576**	0.9416	0.8665	0.7022	0.9254	2.8811	2.7681	5.0897	12.8474	11.9762

	类别 5	0.6298	**0.6351**	0.5847	0.5732	0.3040	1.9538	0.7305	0.8943	2.1995	9.7517
商品一级分类	类别 1	**0.9166**	0.8412	0.8450	0.8142	0.8727	4.8969	8.1522	8.0219	9.1089	12.4504
	类别 2	0.9541	**0.9549**	0.7798	0.7189	0.8219	2.0891	1.7577	7.2852	10.5636	103.0375
	类别 3	0.7922	**0.8088**	0.7070	0.702	0.5322	1.9268	1.3005	1.7243	2.8260	25.6239
商品二级分类	类别 1	**0.9384**	0.8667	0.8163	0.8039	0.8851	3.8935	6.3905	8.3513	9.7088	10.6806
	类别 2	0.9667	**0.9681**	0.8317	0.7709	0.8141	1.4344	1.3744	5.3125	7.9500	12.9854
	类别 3	0.7619	**0.7829**	0.6446	0.6570	0.4993	1.8878	1.2696	1.8020	2.7629	28.3575
	类别 4	**0.9245**	0.8984	0.5880	0.8009	0.8371	3.0316	2.9967	10.2036	6.0462	9.9205
商品三级分类	类别 1	0.7807	0.6252	0.6298	0.7438	**0.8170**	7.6614	12.4993	12.1666	9.9546	9.7974
	类别 2	0.7592	**0.7748**	0.6218	0.7067	0.4924	1.8512	1.1853	1.8102	2.2477	28.2107
	类别 3	**0.9613**	0.9519	0.8565	0.7525	0.8946	2.3173	2.5553	5.7816	11.0620	9.2232
	类别 4	**0.9401**	0.9193	0.7203	0.7584	0.9136	2.0580	2.4157	7.5645	7.2030	6.0436
商家分类	类别 1	**0.9479**	0.9120	0.8540	0.8159	0.8138	3.1818	4.7265	6.5790	8.3305	13.3033
	类别 2	**0.9334**	0.8968	0.7442	0.8222	0.8901	1.9781	2.7678	5.9227	4.4713	8.3497
	类别 3	0.7922	0.8088	0.7070	0.7020	**0.8350**	1.9268	1.3005	1.7243	2.8260	11.9548
	类别 4	**0.9606**	0.9560	0.7738	0.7779	0.8969	2.1822	1.8856	7.9513	9.2250	8.1652
库存分类	类别 1	0.8762	0.8710	0.8321	0.7656	**0.8886**	2.2821	1.3868	1.5677	3.7032	12.2229
	类别 2	**0.9584**	0.9512	0.8942	0.7305	0.8610	1.7716	1.4814	2.8839	8.4323	7.6353
	类别 3	0.8944	0.9040	**0.9123**	0.6705	0.5268	2.8595	2.4452	2.4742	9.8862	27.0617
	类别 4	**0.8968**	0.8834	0.6222	0.8529	0.8882	3.7815	3.5827	9.6849	5.1024	9.6006
商家规模	Large	**0.9183**	0.9181	0.8911	0.6802	0.7928	2.3097	1.4875	1.7610	7.1659	36.6812
	Medium	**0.9543**	0.9159	0.8458	0.8875	0.9112	2.1395	2.6708	4.4599	4.0573	7.1379
	Small	**0.9345**	0.9286	0.7998	0.9005	0.8554	2.2313	2.5315	5.2862	3.3347	9.1847
	Special	0.8192	0.6198	0.5230	0.8280	**0.8282**	4.4013	8.8819	10.8039	4.2843	10.4215
仓库类别	中心仓	**0.9261**	0.9190	0.8862	0.7248	0.7828	2.2625	1.4305	1.8714	6.5381	40.1806
	区域仓	**0.9388**	0.9278	0.8966	0.7654	0.8209	2.1593	1.7124	2.0898	6.0806	10.7697
仓库区域	类别 1	**0.9329**	0.9272	0.8985	0.8308	0.8345	1.8766	1.3808	1.7912	4.0763	10.7157
	类别 2	**0.9306**	0.9259	0.8879	0.7157	0.8131	2.1638	1.3579	1.8335	6.7081	11.6326
	类别 3	**0.9289**	0.8865	0.7888	0.8501	0.9128	3.8721	5.0272	8.4665	6.8619	8.5619

评价指标		MAPE					RMSE				
		LightGBM	ARIMA	EM	XGBoost	LSTM	LightGBM	ARIMA	EM	XGBoost	LSTM
数理特征	类别 1	17.87	17.07	21.37	22.66	32.92	969.57	902.37	904.14	1125.16	5488.83
	类别 2	10.83	13.32	12.11	13.34	20.40	708.90	805.97	863.61	1016.83	8148.83
	类别 3	28.16	56.45	48.23	58.42	32.12	166.09	358.27	308.29	359.53	9255.03
	类别 4	4.03	5.55	13.09	29.54	12.41	1164.74	1470.36	2703.55	5487.15	9381.22

	类别 5	41.05	40.49	53.49	47.67	139.78	447.64	421.97	516.64	477.66	6550.42
商品一级分类	类别 1	7.81	15.13	14.76	18.10	20.66	413.72	695.07	683.97	756.35	3160.76
	类别 2	4.61	4.48	21.71	28.04	24.34	922.84	955.10	3958.66	4862.54	17378.10
	类别 3	20.01	16.77	32.35	28.79	63.30	2032.82	2163.20	2868.02	2887.99	22699.74
商品二级分类	类别 1	5.87	12.82	17.85	19.35	17.81	571.40	1068.17	1395.92	1438.68	4893.00
	类别 2	3.32	3.12	16.63	22.77	25.32	433.53	483.93	1870.47	2469.21	10954.27
	类别 3	23.23	19.27	40.90	34.06	61.71	1975.65	2093.97	2972.03	2793.62	22678.32
	类别 4	7.54	9.99	40.60	19.72	20.54	459.15	578.66	1970.29	1000.26	4127.42
商品三级分类	类别 1	19.25	32.88	32.68	20.96	12.84	337.77	510.55	496.96	416.54	921.70
	类别 2	23.20	20.48	44.19	30.27	66.07	1849.54	1895.18	2894.42	2170.53	17222.74
	类别 3	3.76	4.59	14.12	24.54	14.30	660.25	886.82	2006.53	3270.88	8202.16
	类别 4	5.98	7.99	27.57	24.06	12.17	582.74	808.22	2530.87	2134.82	4607.71
商家分类	类别 1	4.91	8.26	14.13	18.10	23.45	401.30	639.11	889.60	1062.58	10239.67
	类别 2	6.56	10.42	25.28	17.69	12.83	471.18	742.79	1589.49	1081.31	9064.97
	类别 3	20.01	16.77	32.35	28.79	16.15	2032.82	2163.20	2868.02	2887.99	304.65
	类别 4	3.93	4.29	22.18	22.29	11.82	479.29	545.83	2301.66	2155.17	3177.66
库存分类	类别 1	11.04	11.53	16.79	21.76	17.20	1836.02	1899.52	2147.33	2879.92	4669.14
	类别 2	4.11	4.78	10.45	26.83	18.94	734.77	832.70	1621.01	3734.82	4384.15
	类别 3	10.88	10.06	8.90	33.45	67.02	36.54	31.42	31.79	124.94	23973.51
	类别 4	10.53	11.19	36.85	14.48	17.96	561.22	662.68	1791.40	840.86	6574.14
商家规模	Large	7.83	7.77	10.58	31.23	26.70	2671.31	2736.20	3239.18	8313.93	30451.76
	Medim	4.52	8.11	15.10	10.95	14.70	77.45	113.79	190.01	147.85	623.50
	Small	6.29	6.58	19.45	9.44	18.96	126.83	159.47	333.00	189.27	1287.88
	Special	16.36	33.52	43.02	15.83	19.51	336.37	678.49	825.03	321.01	1639.46
仓库类别	中心仓	7.24	7.96	11.11	26.91	27.19	1755.00	1797.14	2351.02	5137.40	20918.57
	区域仓	5.85	6.81	10.15	23.07	20.19	960.25	1095.87	1337.40	2701.69	12372.48
仓库区域	类别 1	6.52	7.00	9.93	16.30	21.71	668.02	701.90	910.49	1446.37	8757.07
	类别 2	6.68	7.21	10.97	27.87	19.58	1821.48	1839.51	2483.74	5705.42	22442.78
	类别 3	6.79	10.70	20.38	14.59	16.00	207.21	299.32	504.10	368.83	1263.04

LightGBM 在 9 个分类标准下和集成后的预测输出与真实结果如图 5.12 所示。从图中可以看出，在不同的分类标准下，9 个预测输出序列表现了相似的走势特征。但是，由于类别划分和训练模型时所用特征集的不同，9 个预测输出序列的变化区间存在差异。为了克服单模型泛化能力弱与不稳定等问题，对 9 个分类标准下的预测值进行集成作为最终的预测输出。集成后的预测值有效地平滑了各模型的缺陷，获得了更小的 WAMAPE、RMSSE 指标。

基于上述分析，采用图 5.5 中的流程框架和 LightGBM 算法对附件中各商品在 2023/05/16-2023/05/30 期间的需求量进行预测，相应结果已填写至附件中的表 1。

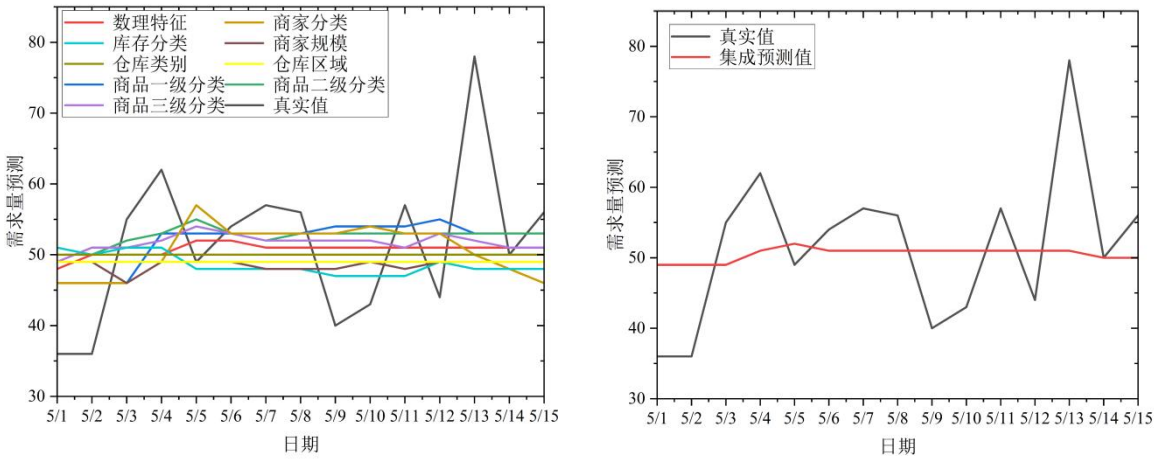


图 5.12 各分类标准下以及集成后的预测输出与真实结果

6 问题二模型建立与求解

问题二主要涉及预测任务。相对于问题一，主要是出现了新的商家+仓库+商品维度，这些实例主要有以下特征：1）出货量序列信息不一致，部分实例由于是新上市或改变了原先存放的仓库，导致不同序列长度不一致；2）出现了新的第一层级类别，使得难以直接借助问题一的架构进行分类和预测。

针对上述问题，本文提出了一种融合 K-means 聚类的多模型集成预测算法。首先，将附件 1 与附件 5 的数据整合（1996 条+210 条），对 2023/4/23-2023/5/15 期间所有数据提取数理特征，并与独热编码后的 8 个解释变量特征（商品一级类别等）汇整，构建分类特征集合，采用 K-means 算法对所有实例进行聚类，将 2206 条序列分为 14 类。然后，采用滑动窗口对聚类后各类别序列进行特征提取和样本集构建，采用支持向量机（SVM）、XGBoost、LightGBM、LSTM、KNN 算法分别训练预测模型，基于 5 折交叉验证法调整模型参数并评估算法性能。在预测阶段，首先对待预测实例进行分类，基于该类别中 5 个算法模型得到 5 个不同的预测值，并将各模型预测值进行集成得到最终的预测输出，整体流程如图 6.1 所示。

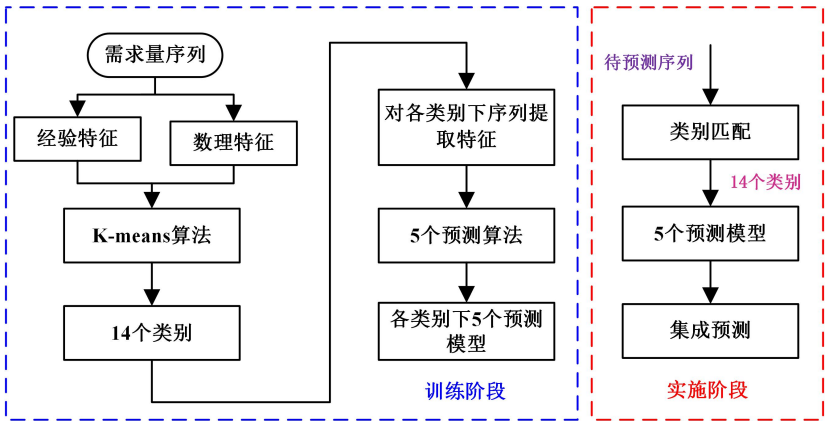


图 6.1 问题 2 框架的训练与实施流程

6.1 基于融合特征的单层聚类算法

6.1.1 特征提取

根据附件 1 和附件 5 中的数据，首先构建由商家-商品-仓库形成的需求量序列，共计 2206 条时间序列实例。由于出现了新上市的商品或改变了某些商品所放仓库未知，导致不同实例的需求量序列长度不一致。 针对该问题，对附件信息进行分析整理，截取商家-仓库-商品序列在 2023/4/23-2023/5/15 期间的数据，保证 2206 条序列长度一致。与问题一不同，问题二出现了新的第一层级类别，无法使用先验字符段特征进行初步分类。为克服该问题，采用独热编码对字符段特征进行量化表示，将这些特征与数理特征堆积共同用于抽象聚类。具体地，独热编码适用于特征取值离散且无实际意义的场景，采用 0 和 1 进行处理，以商家规模为例，编码结果如图 6.2 所示。

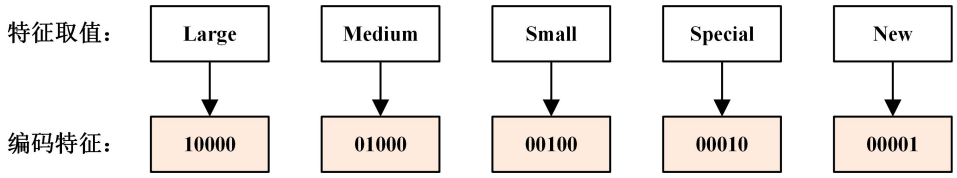


图 6.2 独热编码示意图

为了捕获需求量序列的周期性、季节性等特征，同时考虑在实际商品销售时，相邻日期出货量的强相关性，对序列进行划分。首先，以一周为周期对每条需求量序列进行划分得到 3 段长度为 7 的新序列。针对每段新序列，提取如下的聚类特征：平均值、标准差、最大值、最小值、趋势性强度、峰度和偏度，共 21 个特征。然后对原始序列提取傅里叶变换后的主频分量作为附加特征，与片段特征堆积得到维度为 2206*22 的特征集。时域信号的傅里叶变换可以表示为：

$$X(f)=\int_{-\infty}^{\infty}x(t)\bullet e^{-j2\pi ft}dt,\ \ A(f)=\left|X(f)\right| \qquad (6\text{-}1)$$

式中，$X(f)$ 为频域表示，$x(t)$ 是时域信号，f 是频率。$A(f)$ 为频率 f 处的振幅。在幅度谱中，寻找最大的振幅或能量对应的频率分量。

6.1.2 K-means 聚类算法构建

基于上述构造的特征集，采用 K-means 算法对 2206 条序列进行聚类，基于肘部法则和轮廓系数评估聚类结果，最优 k 值的寻优过程如图 6.3 所示。可以看出，当 $k=14$ 左右时，轮廓系数开始收敛，簇内平方和达到了适中的量级。算法执行流程如表 6.1 所示。

表 6.1 K-means 聚类算法流程

算法 1：K-means 聚类算法

输入：类别数量 k，最大迭代次数 $t_{\max}$，质心变化阈值 δ

选择 k 个随机样本作为初始质心

while $i<i_{\max}$ or 质心变化量$<\delta$

分配样本到最近的质心，并计算每个聚类的平均值

更新质心，并计算当前质心与更新后质心的差距

$i=i+1$

end while

输出：聚类结果

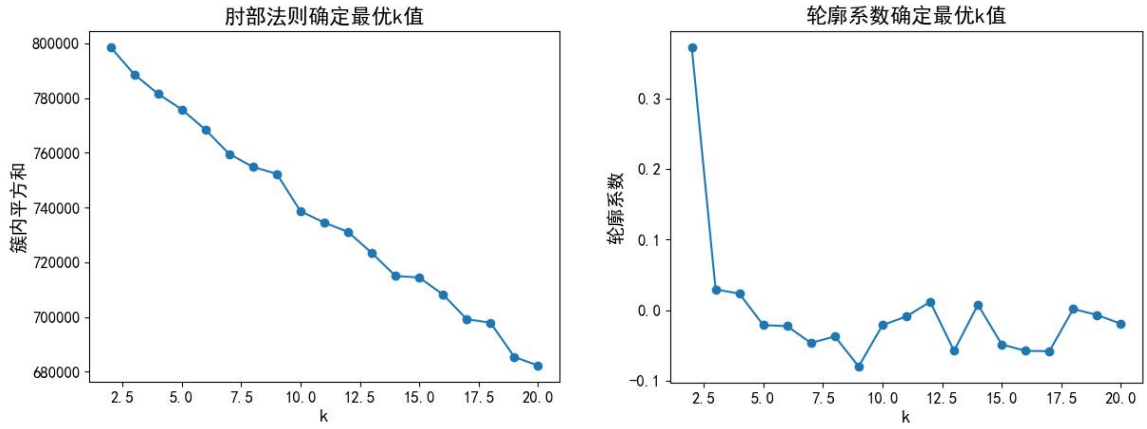


图 6.3 肘部法则和轮廓系数选择最优的聚类数量

6.1.3 聚类结果评估

基于 K-mans 算法，采用章节 6.1.1 中所述的几类特征指标对附件 1 和附件 5 中的 2206 条实例进行聚合。将聚类后的结果使用主成分分析（PCA）降维并可视化聚类结果，结果如图 6.4 和 6.5 所示。从图中可以看出，本文构建的聚类方案可以较好地挖掘出序列中类分离的显著特征。各簇边界明显且簇类中心点相互远离，使同一类别在需求上的特征最为相似。

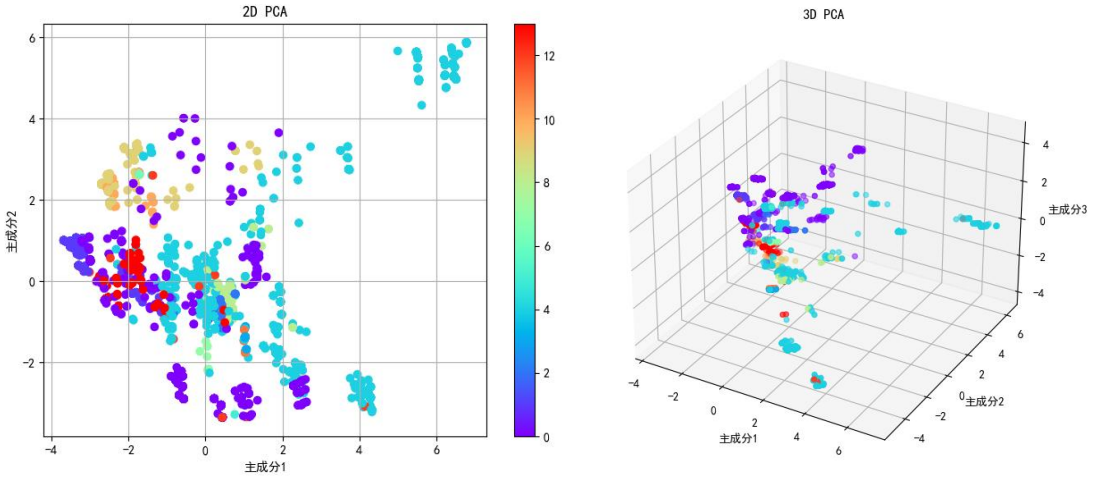
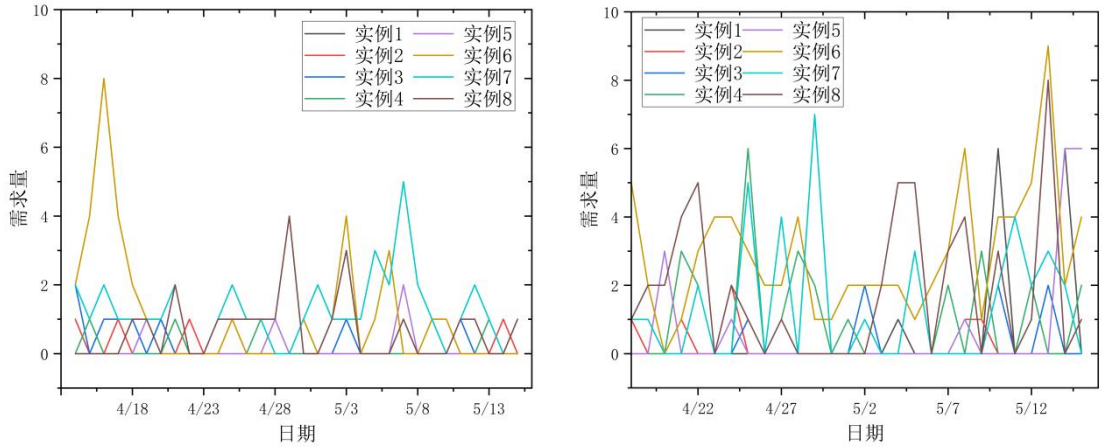


图 6.4 K-means 聚类后的可视化分类图



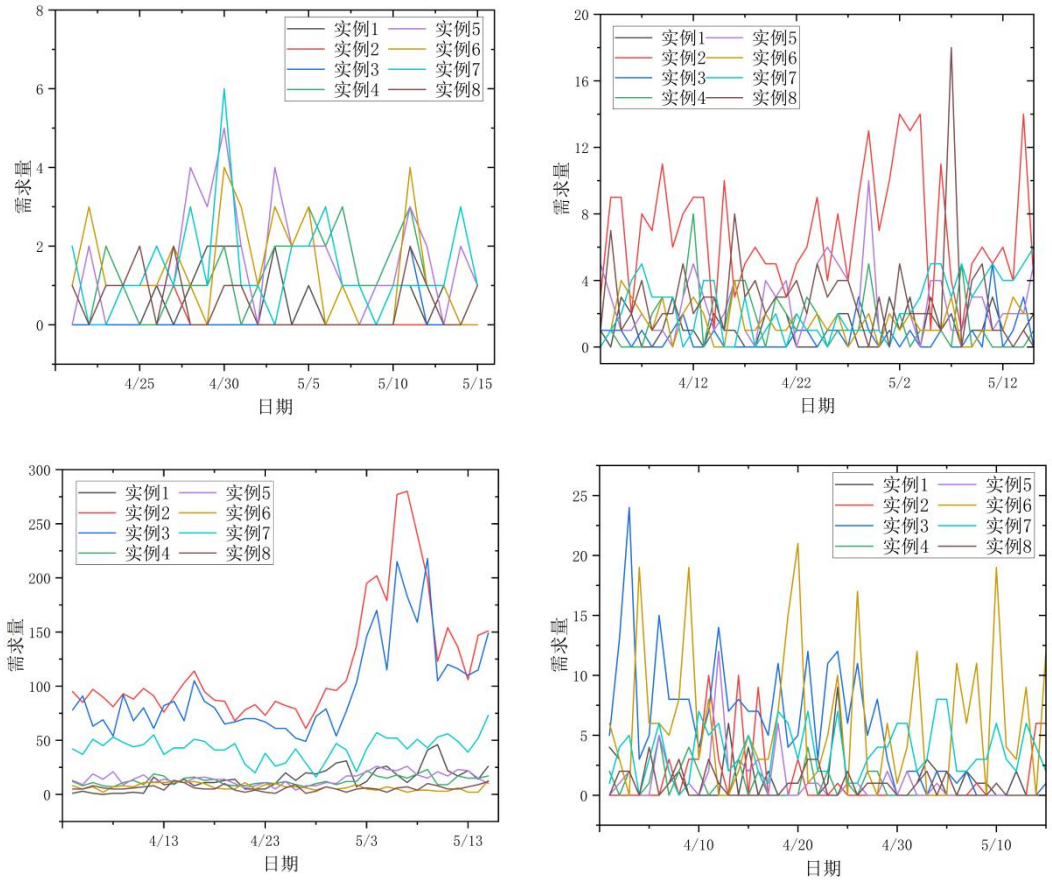


图 6.5 分类标准 1-6 下序列的分布情况

6.2 多模型集成预测算法

在获得聚类标准后，对 2206 条序列进行分类。不同于问题一，问题二不对各类别中的序列数据进行加和，避免各样例的个性化特征被淹没。采用滑动窗口对聚类后各类别中的序列依次进行特征提取和样本集构建。比如，若类别 1 中具有样本量 112 条，每个序列提取 20 个特征，则构造的数据集维度为 112*20。然后，对于各类别分别采用 SVM、XGBoost、LightGBM、LSTM、KNN 算法训练预测模型（各算法下均训练 14 个模型），基于 5 折交叉验证法调整模型参数并评估算法性能。在预测阶段，首先对待预测实例进行分类，基于该类别中 5 个算法模型得到 5 个不同的预测值，并将各模型预测值进行集成得到最终的预测输出，以类别 1 中的序列预测任务为例，实施流程如图 6.6 所示。

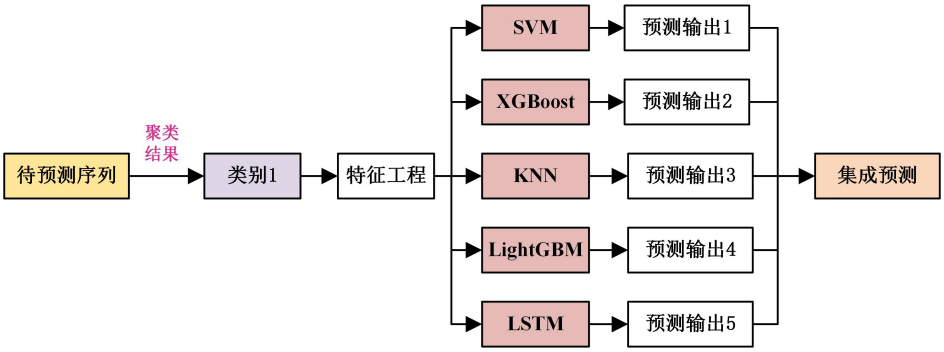


图 6.6 类别 1 中需求量序列预测任务流程

6.2.1 预测特征提取

合理的特征工程会大幅提高模型的效果。根据聚类结果匹配附件 1 和附件 5 中 2022/12/01-2023/5/15 期间的需求量序列，设置滑动窗口长度为 15，截取长度为 15 的序列作为输入样本，该序列后一天的需求量为输出标签，通过滑动窗口构建最终的训练集合。在聚类阶段，由于附件 1 和附件 5 中不同实例的需求量序列长度不一致，我们采取了等维分割取 2023/4/23-2023/5/15 期间的数据进行分类。而在预测模型构建阶段，为了充分地利用数据量，采用滑动窗口构建切割序列集合，克服了不同序列长度不一致的问题。考虑到预测模型强大的挖掘能力，采用基于滑动窗口构造的数据集直接训练 SVM、XGBoost、LSTM 和 KNN 预测模型。

针对基于决策树的 LightGBM 算法，需要进一步挖掘数理特征以保证预测任务的实施。为了捕获需求量序列的周期性、季节性、滞后性等特征，同时考虑在实际商品销售时，相邻日期出货量的强相关性，对于每个切割序列，构建如下的特征：各切割序列倒序 1 天到倒序 7 天的需求量之和；倒序 7 天、14 天的需求量均值和标准差；切割序列的最大值和最小值；切割序列的趋势性强度；周期性/趋势性强度，共计 6 个特征。

6.2.2 预测算法构建

针对聚类后的 14 个类别，分别采用 SVM、XGBoost、LightGBM、LSTM、KNN 算法训练预测模型。其中，XGBoost、LightGBM、LSTM 算法原理与构建流程与问题一相似，具体见章节 5.2。SVM 和 KNN 算法的原理和执行流程如下：

1）基于 SVM 的预测模型构建

支持向量机（SVM）的基本思想是将数据映射到高维空间中，并在该空间中找到一个超平面，使得各类数据点到该超平面的距离最大。具体来说，对于给定的训练数据集，SVM 会通过计算每个样本点与超平面之间的距离，进而确定最佳的决策边界。为了避免过拟合和提高泛化性能，SVM 引入了核函数，可以将线性不可分的数据映射到高维空间，从而实现非线性分类。训练任务可以表示为：

$$\min \ \frac{1}{2}\|W\|^2, \quad \text{s.t.} \ \ y_i(W^T x_i + b) \geq 1, \ i = 1, \ 2, \ ..., \ N \tag{6-2}$$

式中，$y_i(\bullet)$ 为分类超平面。采用构建的切割序列集优化上述问题得到最终的预测模型。

2）基于 KNN 的预测模型构建

K 近邻算法是一种常用的回归方法。基本思想是对于给定的样本，在训练数据集中寻找与它最近的 K 个邻居，通过 K 个邻居的信息来预测这个样本的数值。训练任务可以表示为：

$$\begin{gathered} y = \arg\max_{c_j} \sum_{x_i \in N_k(x)=1} I(y_i = c_j) \\ i = 1, \ 2, \ ..., \ N; \ j = 1, \ 2, \ ..., \ K \end{gathered} \tag{6-3}$$

式中，x_i 为实例的特征向量，y_i 为实例的标签，$I(\bullet)$ 为指示函数。采用构建的切割序列集来训练 KNN 模型，计算样本与训练数据的相似度，筛选最近邻的 K 个样本，采用 K 个邻居标签的平均值作为序列的预测值，进而实现预期的预测任务。

6.3 任务预测与性能评价

基于附件 1 和 5 中商品-商家-仓库在 2022/12/01-2023/5/15 期间形成的需求量序列和图 6.1 的框架，分别采用 SVM、XGBoost、LightGBM、LSTM、KNN 算法训练预测模型，各算法的拟合能力和在验证集上的表现如图 6.7 所示。从图中可以看出，提出的各算法均可以挖掘出时序数据中的抽象特征，实现对出货量的预测。集成后的预测值克服了单模型泛化能力弱的问题，有效地平滑了各模型的缺陷，可以更好地逼近真实值。

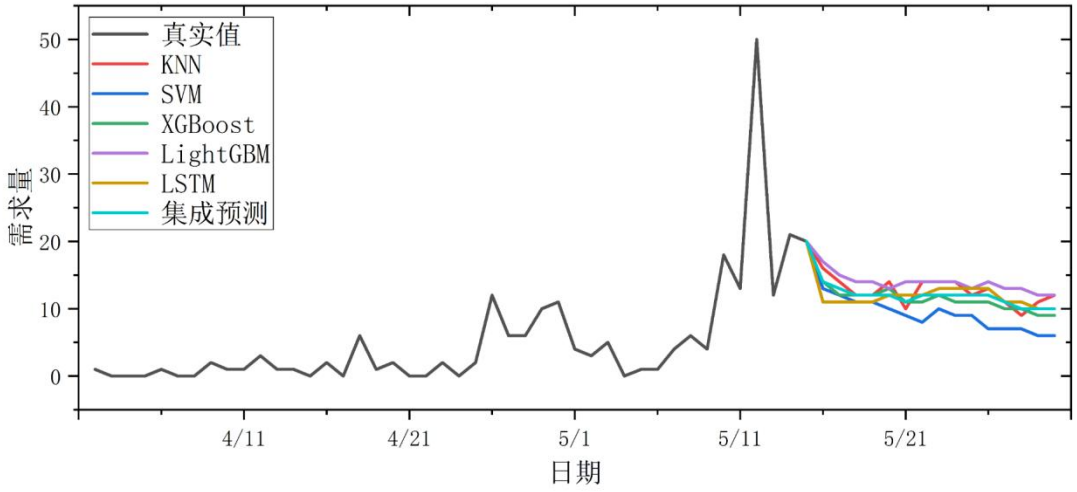


图 6.7 某实例在各算法下以及集成后的预测输出与真实结果

针对各类算法，采用 5 折交叉验证法进行模型训练与性能评估。选取 1-wmape、MAPE 和 RMSE 指标评价各算法优劣，结果如表 6.2 所示。从表中看出，由于各类别下序列分布的数理特征存在差异，同种算法在不同类别下展现了互异的预测性能。比如，在指标 1-WAMAPE 下，KNN 模型在类别 3、7、8、13 中获得了最好评价，SVM 模型在类别 1、4、6 中获得了最好评价，XGBoost 在类别 12 上获得最好评价，LightGBM 在类别 2、5 上获得了最好评价，LSTM 在类别 9、10、11、14 上获得了最好评价。因此，在实际预测任务中，电商平台可以在给定指标下对于每个类别选取最优的模型进行预测。基于上述分析，采用图 6.1 中的算法架构对附件 5 中各商品在 2023/05/16-2023/05/30 期间的需求量进行预测，相应结果已填写至附件中的表 2。

表 6.2 不同算法下预测性能评价

五折交叉验证平均指标	1-WMAPE					MAPE				
	KNN	SVM	XGBoost	LightGBM	LSTM	KNN	SVM	XGBoost	LightGBM	LSTM
类别 1	0.3691	**0.3801**	0.3618	0.3744	0.3686	743.0695	**603.0286**	833.8921	762.4272	844.0827
类别 2	0.5762	0.5845	0.5895	**0.5914**	0.5887	349.2767	**306.6765**	424.9912	367.4352	429.6076
类别 3	**0.5306**	0.4916	0.5532	0.5040	0.5028	334.8504	394.5991	**324.5619**	312.2125	470.2088
类别 4	0.5318	**0.5414**	0.5278	0.4778	0.5562	249.0347	251.4863	268.2551	272.1805	**241.0364**
类别 5	0.6240	0.5945	0.6365	**0.6441**	0.6431	293.1223	**263.2717**	387.4426	303.1533	331.3930

类别 6	0.7026	**0.7240**	0.7184	0.6865	0.6587	39.7623	38.0004	**37.7473**	42.3758	35.4796
类别 7	**0.5556**	0.2051	0.4583	0.5231	0.3566	**1516.7200**	3029.2272	2089.5253	1738.4156	1981.6913
类别 8	**0.3643**	0.3522	0.3693	0.2311	0.3648	284.9975	**219.1434**	335.5463	365.6069	342.4850
类别 9	0.5138	0.4997	0.5225	0.5419	**0.5625**	296.7115	**277.3866**	342.2849	308.6810	359.6722
10	0.4858	0.4757	0.4958	0.5019	**0.5316**	555.7322	**460.9982**	619.9490	547.2010	649.9891
11	0.3735	0.3687	0.3763	0.3693	**0.3992**	487.2293	**398.3599**	578.7195	520.9615	526.2898
12	0.4245	0.3960	**0.4556**	0.4370	0.4297	388.0059	**341.6599**	395.3457	399.0321	360.2626
13	**0.5435**	0.5229	0.5285	0.5212	0.4779	1536.7208	1971.2279	1934.4179	**1513.0256**	2403.5921
14	0.5213	0.3865	0.5551	0.5346	**0.5986**	322.8405	**290.1643**	360.4213	322.9552	305.4850

评价指标		类别 1	类别 2	类别 3	类别 4	类别 5	6	7	8	9	10	11	12	13	14
RMSE	KNN	11.2279	8.0715	**7.7664**	6.2921	8.1224	13.1281	325.2324	4.6855	14.8003	27.3384	5.8995	5.1284	**83.0691**	11.1947
	SVM	11.9593	8.3930	8.9299	6.4535	11.1154	**12.5144**	419.3711	5.0289	16.0401	28.5039	6.6965	5.7457	86.6652	18.3486
	XGBoost	11.3804	7.6395	7.3462	6.1802	7.7060	12.6448	**319.2013**	4.5505	14.9167	26.2668	5.8070	4.8282	86.3768	10.0020
	LightGBM	11.5449	7.8339	8.2468	6.7877	7.6340	13.8704	326.2910	5.1463	13.5937	25.8895	5.9375	4.9541	88.0488	10.6524
	LSTM	**11.0880**	**7.6021**	7.8458	**5.9545**	**7.3311**	15.8262	433.8531	**4.5657**	**10.0356**	**17.4091**	**5.5168**	**4.7861**	89.7018	**7.7281**

7 问题三模型建立与求解

7.1 问题分析与模型建立

问题三主要是考虑在出现规律性大型促销活动时，对实例在 2023/06/01-2023/06/20 期间的需求量进行预测。根据实际分析，将 2023/06/01-2023/06/20 期间活动峰值确定在 2023/06/18（京东 618）。大型促销活动期间，商品的需求量将出现以下特点：1）序列无序性增强：在促销活动期间，商品出货量通常会经历销售峰值，出货量显著增加，商品的库存快速减少，导致序列的规律性较常规期间大幅降低；2）样本稀少：规律性的促销活动只在一段时间内定期出现，相应的历史数据较为稀少，样本的数理特征难以挖掘和关联。

针对上述问题，首先根据附件 1 中 1996 件商品在 2022/12/01-2022/12/11、2023/01/01-2023/01/11 、 2023/02/01-2023/02/11 、 2023/03/01-2023/03/11 、2023/04/01-2023/04/11 这 5 个月 1 日至 11 日的需求量，计算各商品每日均值，得到附件 1 中每件商品在 1 日至 11 日的需求量均值，作为该商品在非促销期间的每日销售量。则第 i 个商品在第 j 日的非促销期间每日平均销售量 $y_{\mathrm{cm}_(i,j)}$ 的计算公式为：

$$y_{\mathrm{cm}_(i,j)}=\frac{1}{5}\sum_{m=1}^{5}y_{\mathrm{cm}_(m,i,j)} \tag{7-1}$$

式中，$i=1,\ 2,\ ...,\ 1996$，$j=1,\ 2,\ ...,\ 11$。$y_{\mathrm{cm}_(m,i,j)}$ 为第 i 个商品在第 m 月第 j 日的销售

量。

然后将附件 6 中 1996 件商品每件商品在 2022/11/01-2022/11/11 这 11 天的销售量作为该商品在促销期间的每日销售量 $y_{\text{act}_(i,j)}$。随后计算每个商品在 1 日至 11 日的促销期间每日销售量与非促销期间每日销售量的比值，得到每一件商品在 1 日至 11 日的促销力度值 $P_{(i,j)}$，计算公式为：

$$P_{(i,j)}=\frac{y_{\text{act}_(i,j)}}{y_{\text{cm}_(i,j)}},\quad i=1,\ 2,\ ...,1996;\ j=1,\ 2,\ ...,\ 11 \tag{7-2}$$

问题三需要预测的促销期间每日销售量等于模型基于非促销期间每日销售量的预测值与促销力度值的乘积，需要计算出各商品每日对应的促销力度值。由于问题三需要预测出附件 1 中 1996 件商品每个商品在 2023/06/01-2023/06/20 期间 20 天的商品需求量，不能与根据附件 6 计算得到的每一件商品在 1 日至 11 日的 11 个促销力度值一一对应，因此将根据附件 6 计算得到的每一件商品在 1 日至 11 日的促销力度值划分为 3 组，分别为 1 日-5 日、6 日-10 日、11 日。计算每组的促销力度值均值，计算方法如下：

$$P_{(i)}^{1}=\frac{1}{5}\sum_{j=1}^{5}P_{(i,j)},\ P_{(i)}^{2}=\frac{1}{5}\sum_{j=6}^{10}P_{(i,j)},\ P_{(i)}^{3}=P_{(i,10)} \tag{7-3}$$

同时将问题三需要预测的 1996 件商品在 2023/06/01-2023/06/20 期间 20 天的商品需求量分为 4 组，分别为 2023/06/01-2023/06/09、2023/06/10-2023/06/17、2023/06/18、2023/06/19-2023/06/20。根据附件 6 计算得到的每一件商品在 1 日至 11 日的 3 组促销力度值，与问题三需要预测的 1996 件商品在 2023/06/01-2023/06/20 期间 20 天的 4 组商品需求量，其对应关系为：将这 3 组促销力度值的均值分别作为 2023/06/01-2023/06/09、2023/06/10-2023/06/17、2023/06/18 这 3 个时间段的促销力度值，第 4 组（2023/06/19-2023/06/20）的促销力度值根据节后的商业经验判断其应该为 1，即认为其与非促销期间该日每日销售量一致，整体流程如图 7.1 所示。

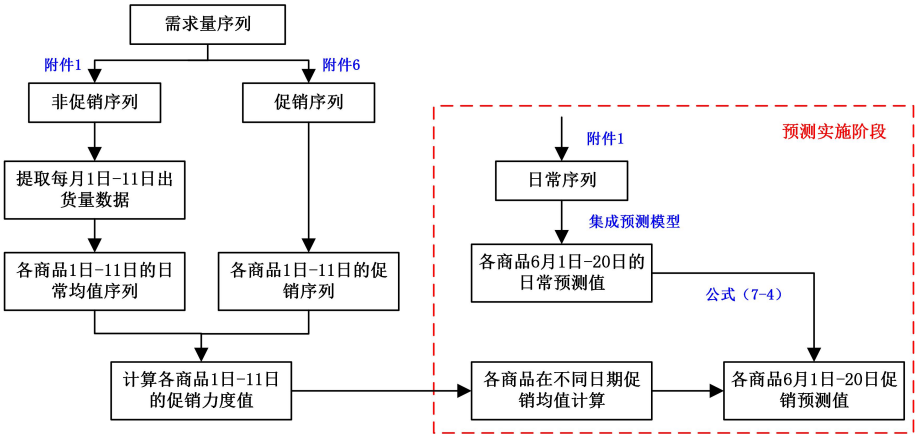


图 7.1 问题 3 框架的训练与实施流程

问题三需要预测的促销期间每日销售量等于模型基于非促销期间每日销售量的预测值与促销力度值的乘积。$y_{\text{pre_cm}_(i,j)}$ 表示问题一中模型预测的第 i 个商品在第 j 日的非促销期间每日销售量，则第 i 个商品在第 j 日的促销期间每日销售量 $y_{\text{pre_act}_(i,j)}$ 可以通过

下式求解：

$$\begin{cases} y_{\text{pre_act}_(i,j)} = y_{\text{pre_cm}_(i,j)} \bullet P_{(i)}^{1}, j = 1,\ 2,\ ...,\ 9 \\ y_{\text{pre_act}_(i,j)} = y_{\text{pre_cm}_(i,j)} \bullet P_{(i)}^{2}, j = 10,\ 11,\ ...,\ 17 \\ y_{\text{pre_act}_(i,j)} = y_{\text{pre_cm}_(i,j)} \bullet P_{(i)}^{3}, j = 18 \\ y_{\text{pre_act}_(i,j)} = y_{\text{pre_cm}_(i,j)}, j = 19,\ 20 \end{cases} \tag{7-4}$$

7.2 预测结果分析

以某一件商品为例，图 7.2 展示了促销力度值作用前后的效果。其中蓝色曲线代表未考虑促销力度值时模型预测出的常规期间商品销量，黄色曲线代表对模型基础预测施加促销因子之后的销量。根据该图可以看出，本文提出的促销事件影响模型对商品的促销期间销量进行了优化，使其更符合促销期间的实际情况。

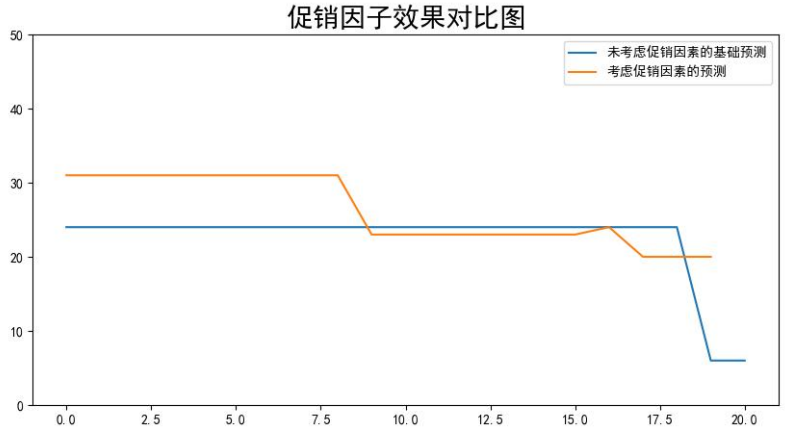


图 7.2 促销因子效果对比图

基于上述分析，采用图 7.1 中的算法架构对附件 6 中各商品在 2023/6/01-2023/06/20 期间的需求量进行预测，相应结果已填写至附件中的表 3。

8 模型评价与推广

8.1 模型的优点

1、针对电商供应链预测任务的特点，设计预测任务的层次结构来实现更好的预测，充分挖掘时序数据特征实现精准预测，**模型具有一定的可解释性**。

2、对数据集进行充分挖掘，建立特征工程，同时考虑了数据的数理特征和解释变量，**对数据集的处理比较完整**。

3、引入**集成学习思想**，综合考虑多模型预测效果，减少极端数据带来的误差，降低个例的随机性对总体预测的影响，多方面学习数据中内在联系，结合多个模型优良方面，避免陷入局部最优。

4、对机器学习模型采用了**交叉验证**的方式，更好地利用数据集，减少数据浪费，提高模型的泛化能力，验证模型的稳健性，**防止过拟合**情况的发生。

5、**模型具有可拓展性**，可以引入更多性能优良的时序预测模型或考虑更多解释变

量，来支撑电商供应链更精准的预测任务。

8.2 模型的缺点

1、特征工程的数理特征提取仍有改进空间，后续可以对特征工程更好的处理。

2、受限于时间，各时序预测模型参数可以进一步寻优。

8.3 模型的推广

精准的需求预测对于电商商家降低成本、保证履约等具有重要意义。本文采用的层次时序预测思想对于稀缺数据、新维度数据等的预测能够起到良好的效果，综合考虑数理特征与外生变量的融合，有利于充分挖掘相似数据模式，指导同一类别数据的预测。另外，本文引入了多种机器学习时序预测模型，基于大数据驱动的方法能够极大方便当今对复杂数据的处理。本文的模型不仅仅能够在供应链优化当中提供整体和局部的需求预测，也有潜力应用于能耗预测、金融市场风险管理、公共健康领域等，利用不同层级的时间序列间的依赖关系和模式进行精准预测，实现良好的效益。

参考文献

[1] 杨博, 罗嘉琛, 宋艳涛, 等. 基于对比学习的时间序列聚类方法[J/OL]. 计算机科学: 1-13 [2023-11-03].

[2] 韩立, 李芳. 基于动态特征选择和长短期记忆模型的气温预报方法[J]. 计算机时代, 2023(10): 148-156.

[3] Adnan R M, Parmar K S, Heddam S, et al. Suspended Sediment Modeling Using a Heuristic Regression Method Hybridized with Kmeans Clustering[J]. Sustainability, 2021, 13. DOI:10.3390/su13094648.

[4] Makridakis S, Spiliotis E, Assimakopoulos V. The M5 Accuracy competition: Results, findings and conclusions[J]. International Journal of Forecasting, 2020, 36(1): 224-227.

[5] Makridakis S, Spiliotis E, Assimakopoulos V, et al. The M4 Competition: 100,000 time series and 61 forecasting methods[J]. International Journal of Forecasting, 2020, 36(1): 54-74.

[6] Cai R, Xie S, Wang B, et al. Wind speed forecasting based on extreme gradient boosting[J]. IEEE Access, 2020, 8: 175063-175069.

[7] 王鹏新, 王颖, 田惠仁, 等. 基于 LightGBM 的冬小麦产量估测及可解释性研究[J/OL]. 农业机械学报: 1-16 [2023-11-03].

[8] 韩克宁, 王伟, 樊冬艳, 等. 基于产量递减与 LSTM 耦合的常压页岩气井产量预测[J]. 油气藏评价与开发, 2023, 13(05): 647-656.

[9] 荣新, 覃卫坚, 韦文山. 基于 3 种机器学习算法的台风频数预测[J]. 海洋预报, 2023, 40(05): 1-9.

[10] 李明宇. 大型商超白酒产品促销效能评价方法研究[J]. 中国酿造, 2023, 42(09): 246-252.

附录

No: 1	Filename: p1-category8-Kmeans.ipynb

```
import pandas as pd
import numpy as np
from scipy.stats import linregress, skew, kurtosis
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

df1 = pd.read_csv("raw/1.csv")
df2 = pd.read_csv("raw/2.csv")
df3 = pd.read_csv("raw/3.csv")
df4 = pd.read_csv("raw/4.csv")

# 合并数据
merged_data = df1.merge(df2, on='product_no', how='left')
merged_data = merged_data.merge(df3, on='seller_no', how='left')
merged_data = merged_data.merge(df4, on='warehouse_no', how='left')

print(merged_data)

group_basis_list = ['category1', 'category2', 'category3', 'seller_category', 'inventory_category', 'seller_level',
'warehouse _category', 'warehouse _region']
group_index = 7
group_basis = group_basis_list[group_index]

# 针对 category1 进行分组，并将每组中的所有时序数据的 qty 字段求和
grouped_data = merged_data.groupby([group_basis, 'date']).agg({'qty': 'sum'}).reset_index()

# 提取特征
results = []
segments = [
    ('2022-12-01', '2022-12-15'),
    ('2022-12-16', '2022-12-31'),
    ('2023-01-01', '2023-01-15'),
    ('2023-01-16', '2023-01-31'),
    ('2023-02-01', '2023-02-15'),
    ('2023-02-16', '2023-02-28'),
    ('2023-03-01', '2023-03-15'),
    ('2023-03-16', '2023-03-31'),
    ('2023-04-01', '2023-04-15'),
    ('2023-04-16', '2023-04-30'),
    ('2023-05-01', '2023-05-15')
]

for category, group in grouped_data.groupby(group_basis):
    # 针对每一组得到的 166 点的时序数据，按时间排序
    group = group.sort_values(by='date')

    features = []
    for start_date, end_date in segments:
        segment_data = group[(group['date'] >= start_date) & (group['date'] <= end_date)]['qty']

        # 计算特征
        mean_value = segment_data.mean()
```

```
            std_dev = segment_data.std()
            max_value = segment_data.max()
            min_value = segment_data.min()
            trend_strength = linregress(range(len(segment_data)), segment_data.values).slope
            # Calculate skewness and kurtosis, set them to 0 if std_dev is 0
            if std_dev == 0:
                skewness_val = 0
                kurtosis_val = 0
            else:
                skewness_val = segment_data.skew()
                kurtosis_val = segment_data.kurt()

            features.extend([mean_value, std_dev, max_value, min_value, trend_strength, skewness_val, kurtosis_val])

        results.append([category] + features)

# 创建 DataFrame
feature_names = ['mean', 'std_dev', 'max', 'min', 'trend_strength', 'skewness', 'kurtosis']
columns = [group_basis]
for i in range(len(segments)):
    for feature in feature_names:
        columns.append(f"{feature}_{i+1}")
result_df = pd.DataFrame(results, columns=columns)
print(result_df)
result_df.to_csv('result/p1/category_{}.csv'.format(group_basis), index=False)

# 标准化特征
scaler = StandardScaler()
scaled_features = scaler.fit_transform(result_df.iloc[:, 1:])

# 确定 KMeans 算法的最佳 n_clusters 值
# 法 1：肘部法则 (Elbow Method)：该方法涉及运行 k-means 聚类在数据集上多次，每次使用不同的 k 值。每次执行算法都会得到一个与之前的 k 值不同的 SSE（每个点到其聚类中心的平方距离之和）。
# 最终，SSE 值会被绘制出来，我们选择“肘部”处的 k 值。
# 法 2：轮廓系数 (Silhouette Coefficient)：轮廓系数是一个用于评估聚类效果好坏的指标，其值介于-1 和 1 之间。接近 1 的值表示样本与其所在的聚类非常相似，接近-1 的值表示样本与其邻近的聚类更为相似。

# 根据 group_basis 判断是否执行 KMeans 聚类以及确定 k 值的范围
execute_kmeans = True
possible_k_values = range(2, min(len(result_df), 11))  # 例如测试 2 到 10 的 k 值
if group_basis in ['inventory_category', 'seller_level', 'warehouse _category']:
    execute_kmeans = False  # 对这三个特定的 group_basis 不进行 KMeans 聚类
elif group_basis == 'warehouse _region':
    possible_k_values = range(2, 7)  # 对'warehouse _region'，测试 2 到 6 的 k 值

if execute_kmeans:
    wcss = []  # Within-cluster sum of squares
    sil_coefficients = []

    for k in possible_k_values:
        kmeans = KMeans(n_clusters=k, random_state=42).fit(scaled_features)
        wcss.append(kmeans.inertia_)
        sil_coefficients.append(silhouette_score(scaled_features, kmeans.labels_, metric='euclidean'))

    # 肘部法则的可视化
    plt.figure()
    plt.plot(possible_k_values, wcss, '-o')
```

```
    plt.title('Elbow Method For Optimal k')
    plt.xlabel('Number of Clusters (k)')
    plt.ylabel('WCSS')
    plt.show()

    # 轮廓系数的可视化
    plt.figure()
    plt.plot(possible_k_values, sil_coefficients, '-o')
    plt.title('Silhouette Coefficient For Optimal k')
    plt.xlabel('Number of Clusters (k)')
    plt.ylabel('Silhouette Coefficient')
    plt.show()

if execute_kmeans:
    best_k = 3  # 根据之前的分析确定的最佳 k 值，假设最佳 k 值为 3，你可以根据上一个分析来替换此值
    kmeans = KMeans(n_clusters=best_k, random_state=42).fit(scaled_features)
    # 将聚类标签加入到特征数据集中
    result_df['Cluster'] = kmeans.labels_
else:
    best_k = len(result_df)
    # 对于不需要聚类的 group_basis，直接使用该 group_basis 的值作为 Cluster 值
    result_df['Cluster'] = result_df[group_basis].astype(str)

# 创建一个新的 DataFrame，用于保存商家-商品-仓库组合
combination_df = merged_data[['product_no', 'seller_no', 'warehouse_no', group_basis]].drop_duplicates()
combination_df['Cluster'] = -1  # 初始化聚类标签为-1

# 将聚类标签与商家-商品-仓库组合合并
for idx, row in result_df.iterrows():
    category_label = row[group_basis]
    cluster_label = row['Cluster']
    combination_df.loc[combination_df[group_basis] == category_label, 'Cluster'] = cluster_label

# 确认是否所有的记录都有聚类标签
assert not (combination_df['Cluster'] == -1).any(), "Not all records have cluster labels"

# 保存带有聚类标签的新数据
combination_df.to_csv("result/p1/category_{}_clustered_{}.csv".format(group_basis, best_k), index=False)

# result_df['Cluster'] = kmeans.labels_
# result_df.to_csv("result/p1/category_{}_clustered_{}.csv".format(group_basis,best_k), index=False) # 保存带有聚类标签的新数据

# 为了更好地理解聚类的效果，可以使用 PCA 进行降维并可视化聚类结果
# 降维到 2D
pca_2d = PCA(n_components=2)
principal_components_2d = pca_2d.fit_transform(scaled_features)

# 可视化 2D
plt.figure(figsize=(8, 6))
plt.scatter(principal_components_2d[:, 0], principal_components_2d[:, 1], c=kmeans.labels_, cmap='rainbow')
plt.title('2D PCA of Clustered Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar()
plt.grid(True)
plt.show()
```

```
# 降维到 3D
pca_3d = PCA(n_components=3)
principal_components_3d = pca_3d.fit_transform(scaled_features)

# 可视化 3D
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(principal_components_3d[:, 0], principal_components_3d[:, 1], principal_components_3d[:, 2], c=kmeans.labels_, cmap='rainbow')
ax.set_title('3D PCA of Clustered Data')
ax.set_xlabel('Principal Component 1')
ax.set_ylabel('Principal Component 2')
ax.set_zlabel('Principal Component 3')
plt.show()
```

No: 2	Filename: p1-forecast-autu_arima.ipynb

```
import pandas as pd
import numpy as np
import lightgbm as lgb
import pickle
import joblib
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
from pmdarima import auto_arima

group_list = ['category1', 'category2', 'category3', 'seller_category', 'inventory_category', 'seller_level', 'warehouse _category', 'warehouse _region', 'clustered']
group_file = ['result/p1/category_category1_clustered_3.csv', 'result/p1/category_category2_clustered_4.csv', 'result/p1/category_category3_clustered_4.csv',
              'result/p1/category_seller_category_clustered_4.csv', 'result/p1/category_inventory_category_clustered_4.csv',
              'result/p1/category_seller_level_clustered_4.csv', 'result/p1/category_warehouse _category_clustered_2.csv',
              'result/p1/category_warehouse _region_clustered_3.csv','result/p1/category_clustered_5.csv']
group_index = 8

# 1. 数据加载和预处理
clusters = pd.read_csv(group_file[group_index])
raw_data = pd.read_csv('raw/1.csv')
raw_data['date'] = pd.to_datetime(raw_data['date'])
raw_data = raw_data.sort_values(by='date')

# 2. 按照 Cluster 分类
groups = clusters.groupby('Cluster')

# 函数：计算 1-WMAPE
def compute_wmape(y_true, y_pred):
    assert len(y_true) == len(y_pred), "y_true and y_pred must have the same length"
    return 1 - np.sum(np.abs(y_true - y_pred)) / np.sum(y_true) if np.sum(y_true) != 0 else 1 - np.count_nonzero(y_pred)/len(y_true)

def compute_rmsse(y_true, y_pred, y_train):
    # Find the index of the first non-zero demand
    start_index = np.argmax(y_train > 0)
    # If no non-zero demands are found, start from the beginning
    if start_index == 0 and y_train[start_index] == 0:
        start_index = 0
```

```
    # Adjust y_train to consider only periods after the first non-zero demand
    adjusted_y_train = y_train[start_index:]
    numerator = np.sum((y_true - y_pred) ** 2)
    denominator = np.sum((adjusted_y_train[1:] - adjusted_y_train[:-1]) ** 2) / (len(adjusted_y_train) - 1)
    if denominator == 0:
        return np.inf
    return np.sqrt(numerator / denominator)

# 函数：计算 MAPE
def compute_mape(y_true, y_pred):
    y_true_adjusted = np.where(y_true == 0, 0.1, y_true)  # 使用 np.where 将 y_true 中的 0 替换为 0.1
    return np.mean(np.abs((y_true - y_pred) / y_true_adjusted)) * 100

# 函数：计算 RMSE
def compute_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

all_predictions = []
all_actuals = []
all_predictions_dfs = []
# 列表保存每个组合的预测结果的评价指标
all_wmapes, all_rmsses, all_mapes, all_rmses, all_r2s = [], [], [], [], []
# 列表保存每一个大类总需求量预测的评价指标
cluster_wmapes, cluster_rmsses, cluster_mapes, cluster_rmses, cluster_r2s = [], [], [], [], []
for cluster_id, group in groups:
    print(cluster_id)
    print(group)
    cluster_data = pd.DataFrame()
    for _, row in group.iterrows():
        combo_data = raw_data[(raw_data['seller_no'] == row['seller_no']) &
                              (raw_data['product_no'] == row['product_no']) &
                              (raw_data['warehouse_no'] == row['warehouse_no']) &
                              (raw_data['date'] >= '2022-12-01') & (raw_data['date'] <= '2023-05-15')]
        combo_data = combo_data.sort_values(by='date')
        cluster_data = pd.concat([cluster_data, combo_data])

    # 总需求量时间序列
    total_demand_series = cluster_data.groupby('date')['qty'].sum()
    # print(total_demand_series)

    # 取最后 15 天，即 2023/5/1 到 2023/5/15 的数据作为验证集
    X_train, X_val = total_demand_series.iloc[:-15].values, total_demand_series.iloc[-15:].values

    # 使用 auto_arima 进行训练
    model = auto_arima(X_train, start_p=1, start_q=1, max_p=3, max_q=3, start_P=0, max_d=5, seasonal=True,
m=7, trace=True, error_action='ignore', suppress_warnings=True)

    # 对验证集进行预测
    total_demand_pred = model.predict(n_periods=len(X_val))
    cluster_wmapes.append(compute_wmape(X_val, total_demand_pred))
    cluster_rmsses.append(compute_rmsse(X_val, total_demand_pred, X_train))
    cluster_mapes.append(compute_mape(X_val, total_demand_pred))
    cluster_rmses.append(compute_rmse(X_val, total_demand_pred))
    cluster_r2s.append(r2_score(X_val, total_demand_pred))

    model = auto_arima(total_demand_series.values, start_p=1, start_q=1, max_p=3, max_q=3, start_P=0,
max_d=5, seasonal=True, m=7, trace=True, error_action='ignore', suppress_warnings=True)
    # 保存大类预测模型
    joblib.dump(model,
```

```
'saved_models/autoarima/model_category_{}_cluster_{}.pkl'.format(group_list[group_index],cluster_id))
    # 预测 2023/5/16 到 2023/5/30
    future_predictions = model.predict(n_periods=15)

    # 计算每类中各组合的预测结果
    for _, row in group.iterrows():
        combo_data = raw_data[(raw_data['seller_no'] == row['seller_no']) &
                              (raw_data['product_no'] == row['product_no']) &
                              (raw_data['warehouse_no'] == row['warehouse_no']) &
                              (raw_data['date'] >= '2022-12-01') & (raw_data['date'] <= '2023-05-15')]
        combo_data = combo_data.sort_values(by='date')

        # 计算权重，取 2023/4/1 到 2023/4/30 作为验证集权重的计算
        last_15_days = combo_data.iloc[-15:]
        last_days = combo_data.iloc[-45:-15]
        weights = last_days['qty'] / total_demand_series.iloc[-45:-15].values
        avg_weight = weights.mean()
        # print(weights, avg_weight)

        # 计算预测值
        combo_pred_float = total_demand_pred * avg_weight
        combo_pred = np.round(total_demand_pred * avg_weight)  # 使用 np.round 四舍五入

        # 计算当前组合的评估指标
        wmape = compute_wmape(last_15_days['qty'].values, combo_pred)
        rmsse = compute_rmsse(last_15_days['qty'].values, combo_pred, np.array(combo_data['qty']))
        # mape = compute_mape(last_15_days['qty'].values, combo_pred)
        rmse = compute_rmse(last_15_days['qty'].values, combo_pred)
        r2 = r2_score(last_15_days['qty'].values, combo_pred)
        all_wmapes.append(wmape)
        all_rmsses.append(rmsse)
        # all_mapes.append(mape)
        all_rmses.append(rmse)
        all_r2s.append(r2)

        df_pred = pd.DataFrame({
            'seller_no': row['seller_no'],
            'product_no': row['product_no'],
            'warehouse_no': row['warehouse_no'],
            'date': pd.date_range(start="2023-05-1", end="2023-05-15"),
            'predicted_value_float': combo_pred_float,
            'predicted_value': combo_pred,
            '1-WMAPE': [wmape] * 15,
            'RMSSE': [rmsse] * 15,
            # 'MAPE': [mape] * 15,
            'RMSE': [rmse] * 15,
            'R2': [r2] * 15
        })
        all_predictions_dfs.append(df_pred)  # 将当前预测结果加入到汇总列表中
        all_predictions.extend(combo_pred)
        all_actuals.extend(last_15_days['qty'].values)

        # 计算权重，取 2023/4/16 到 2023/5/15 作为未来 15 天预测值权重的计算
        last_days = combo_data.iloc[-30:]
        weights = last_days['qty'] / total_demand_series.iloc[-30:].values
        avg_weight = weights.mean()
        # 使用上面计算的 future_predictions 计算预测值
        combo_pred_future_float = np.array(future_predictions) * avg_weight
        combo_pred_future = np.round(np.array(future_predictions) * avg_weight)
```

```
        df_pred_future = pd.DataFrame({
            'seller_no': row['seller_no'],
            'product_no': row['product_no'],
            'warehouse_no': row['warehouse_no'],
            'date': pd.date_range(start="2023-05-16", end="2023-05-30"),
            'predicted_value_float': combo_pred_future_float,
            'predicted_value': combo_pred_future
        })
        all_predictions_dfs.append(df_pred_future)  # 将未来的预测结果加入到汇总列表中

# 合并所有的预测结果
final_predictions = pd.concat(all_predictions_dfs, ignore_index=True)

# 保存到 CSV 文件
final_predictions.to_csv('result/p1/forecast_autoarima/forecast_autoarima_{}.csv'.format(group_list[group_index]),
index=False)

# 评估模型
print(f"Average 1-WMAPE: {np.mean(all_wmapes)}")
print(f"Average RMSSE: {np.mean(all_rmsses)}")
# print(f"Average MAPE: {np.mean(all_mapes)}")
print(f"Average RMSE: {np.mean(all_rmses)}")
print(f"Average R2: {np.mean(all_r2s)}")
print("total_demand_pred 1-WMAPE:", cluster_wmapes)
print("total_demand_pred RMSSE:", cluster_rmsses)
print("total_demand_pred MAPE:", cluster_mapes)
print("total_demand_pred RMSE:", cluster_rmses)
print("total_demand_pred R2:", cluster_r2s)
```

No: 3	Filename: p1-forecast-es.ipynb

```
import pandas as pd
import numpy as np
import lightgbm as lgb
import pickle
import joblib
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
from pmdarima import auto_arima
from statsmodels.tsa.holtwinters import ExponentialSmoothing

group_list = ['category1', 'category2', 'category3', 'seller_category', 'inventory_category', 'seller_level',
'warehouse _category', 'warehouse _region', 'clustered']
group_file = ['result/p1/category_category1_clustered_3.csv', 'result/p1/category_category2_clustered_4.csv',
'result/p1/category_category3_clustered_4.csv',
              'result/p1/category_seller_category_clustered_4.csv',
'result/p1/category_inventory_category_clustered_4.csv',
              'result/p1/category_seller_level_clustered_4.csv',              'result/p1/category_warehouse
_category_clustered_2.csv',
              'result/p1/category_warehouse _region_clustered_3.csv','result/p1/category_clustered_5.csv']
group_index = 8

# 1. 数据加载和预处理
clusters = pd.read_csv(group_file[group_index])
raw_data = pd.read_csv('raw/1.csv')
raw_data['date'] = pd.to_datetime(raw_data['date'])
raw_data = raw_data.sort_values(by='date')
```

```
# 2. 按照 Cluster 分类
groups = clusters.groupby('Cluster')

# 函数：计算 1-WMAPE
def compute_wmape(y_true, y_pred):
    assert len(y_true) == len(y_pred), "y_true and y_pred must have the same length"
    return 1 - np.sum(np.abs(y_true - y_pred)) / np.sum(y_true) if np.sum(y_true) != 0 else 1 - np.count_nonzero(y_pred)/len(y_true)

def compute_rmsse(y_true, y_pred, y_train):
    # Find the index of the first non-zero demand
    start_index = np.argmax(y_train > 0)
    # If no non-zero demands are found, start from the beginning
    if start_index == 0 and y_train[start_index] == 0:
        start_index = 0
    # Adjust y_train to consider only periods after the first non-zero demand
    adjusted_y_train = y_train[start_index:]
    numerator = np.sum((y_true - y_pred) ** 2)
    denominator = np.sum((adjusted_y_train[1:] - adjusted_y_train[:-1]) ** 2) / (len(adjusted_y_train) - 1)
    if denominator == 0:
        return np.inf
    return np.sqrt(numerator / denominator)

# 函数：计算 MAPE
def compute_mape(y_true, y_pred):
    y_true_adjusted = np.where(y_true == 0, 0.1, y_true)  # 使用 np.where 将 y_true 中的 0 替换为 0.1
    return np.mean(np.abs((y_true - y_pred) / y_true_adjusted)) * 100

# 函数：计算 RMSE
def compute_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

all_predictions = []
all_actuals = []
all_predictions_dfs = []
# 列表保存每个组合的预测结果的评价指标
all_wmapes, all_rmsses, all_mapes, all_rmses, all_r2s = [], [], [], [], []
# 列表保存每一个大类总需求量预测的评价指标
cluster_wmapes, cluster_rmsses, cluster_mapes, cluster_rmses, cluster_r2s = [], [], [], [], []
for cluster_id, group in groups:
    print(cluster_id)
    print(group)
    cluster_data = pd.DataFrame()
    for _, row in group.iterrows():
        combo_data = raw_data[(raw_data['seller_no'] == row['seller_no']) &
                              (raw_data['product_no'] == row['product_no']) &
                              (raw_data['warehouse_no'] == row['warehouse_no']) &
                              (raw_data['date'] >= '2022-12-01') & (raw_data['date'] <= '2023-05-15')]
        combo_data = combo_data.sort_values(by='date')
        cluster_data = pd.concat([cluster_data, combo_data])

    # 总需求量时间序列
    total_demand_series = cluster_data.groupby('date')['qty'].sum()
    # print(total_demand_series)

    # 取最后 15 天，即 2023/5/1 到 2023/5/15 的数据作为验证集
    X_train, X_val = total_demand_series.iloc[:-15].values, total_demand_series.iloc[-15:].values
```

```
    # 使用指数平滑模型
    model = ExponentialSmoothing(X_train, trend="add", seasonal="add", seasonal_periods=7)
    model_fit = model.fit()

    # 对验证集进行预测
    total_demand_pred = model_fit.forecast(len(X_val))
    cluster_wmapes.append(compute_wmape(X_val, total_demand_pred))
    cluster_rmsses.append(compute_rmsse(X_val, total_demand_pred, X_train))
    cluster_mapes.append(compute_mape(X_val, total_demand_pred))
    cluster_rmses.append(compute_rmse(X_val, total_demand_pred))
    cluster_r2s.append(r2_score(X_val, total_demand_pred))

    # 使用指数平滑模型
    model = ExponentialSmoothing(total_demand_series.values, trend="add", seasonal="add",
seasonal_periods=7)
    model_fit = model.fit()
    # 保存大类预测模型
    joblib.dump(model,
'saved_models/es/model_category_{}_cluster_{}.pkl'.format(group_list[group_index],cluster_id))
    # 预测 2023/5/16 到 2023/5/30
    future_predictions = model_fit.forecast(15)

    # 计算每类中各组合的预测结果
    for _, row in group.iterrows():
        combo_data = raw_data[(raw_data['seller_no'] == row['seller_no']) &
                              (raw_data['product_no'] == row['product_no']) &
                              (raw_data['warehouse_no'] == row['warehouse_no']) &
                              (raw_data['date'] >= '2022-12-01') & (raw_data['date'] <= '2023-05-15')]
        combo_data = combo_data.sort_values(by='date')

        # 计算权重，取 2023/4/1 到 2023/4/30 作为验证集权重的计算
        last_15_days = combo_data.iloc[-15:]
        last_days = combo_data.iloc[-45:-15]
        weights = last_days['qty'] / total_demand_series.iloc[-45:-15].values
        avg_weight = weights.mean()
        # print(weights, avg_weight)

        # 计算预测值
        combo_pred_float = total_demand_pred * avg_weight
        combo_pred = np.round(total_demand_pred * avg_weight)  # 使用 np.round 四舍五入

        # 计算当前组合的评估指标
        wmape = compute_wmape(last_15_days['qty'].values, combo_pred)
        rmsse = compute_rmsse(last_15_days['qty'].values, combo_pred, np.array(combo_data['qty']))
        # mape = compute_mape(last_15_days['qty'].values, combo_pred)
        rmse = compute_rmse(last_15_days['qty'].values, combo_pred)
        r2 = r2_score(last_15_days['qty'].values, combo_pred)
        all_wmapes.append(wmape)
        all_rmsses.append(rmsse)
        # all_mapes.append(mape)
        all_rmses.append(rmse)
        all_r2s.append(r2)

        df_pred = pd.DataFrame({
            'seller_no': row['seller_no'],
            'product_no': row['product_no'],
            'warehouse_no': row['warehouse_no'],
            'date': pd.date_range(start="2023-05-1", end="2023-05-15"),
            'predicted_value_float': combo_pred_float,
```

```
            'predicted_value': combo_pred,
            '1-WMAPE': [wmape] * 15,
            'RMSSE': [rmsse] * 15,
            # 'MAPE': [mape] * 15,
            'RMSE': [rmse] * 15,
            'R2': [r2] * 15
        })
        all_predictions_dfs.append(df_pred)  # 将当前预测结果加入到汇总列表中
        all_predictions.extend(combo_pred)
        all_actuals.extend(last_15_days['qty'].values)

        # 计算权重，取 2023/4/16 到 2023/5/15 作为未来 15 天预测值权重的计算
        last_days = combo_data.iloc[-30:]
        weights = last_days['qty'] / total_demand_series.iloc[-30:].values
        avg_weight = weights.mean()
        # 使用上面计算的 future_predictions 计算预测值
        combo_pred_future_float = np.array(future_predictions) * avg_weight
        combo_pred_future = np.round(np.array(future_predictions) * avg_weight)

        df_pred_future = pd.DataFrame({
            'seller_no': row['seller_no'],
            'product_no': row['product_no'],
            'warehouse_no': row['warehouse_no'],
            'date': pd.date_range(start="2023-05-16", end="2023-05-30"),
            'predicted_value_float': combo_pred_future_float,
            'predicted_value': combo_pred_future
        })
        all_predictions_dfs.append(df_pred_future)  # 将未来的预测结果加入到汇总列表中

# 合并所有的预测结果
final_predictions = pd.concat(all_predictions_dfs, ignore_index=True)

# 保存到 CSV 文件
final_predictions.to_csv('result/p1/forecast_es/forecast_es_{}.csv'.format(group_list[group_index]),
index=False)

# 评估模型
print(f"Average 1-WMAPE: {np.mean(all_wmapes)}")
print(f"Average RMSSE: {np.mean(all_rmsses)}")
# print(f"Average MAPE: {np.mean(all_mapes)}")
print(f"Average RMSE: {np.mean(all_rmses)}")
print(f"Average R2: {np.mean(all_r2s)}")
print("total_demand_pred 1-WMAPE:", cluster_wmapes)
print("total_demand_pred RMSSE:", cluster_rmsses)
print("total_demand_pred MAPE:", cluster_mapes)
print("total_demand_pred RMSE:", cluster_rmses)
print("total_demand_pred R2:", cluster_r2s)
```

No: 4	Filename: p1-forecast-lightgbm.ipynb

```
import pandas as pd
import numpy as np
import lightgbm as lgb
import pickle
import joblib
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
```

```
WINDOWS_SIZE = 30
group_list = ['category1', 'category2', 'category3', 'seller_category', 'inventory_category', 'seller_level', 'warehouse
_category', 'warehouse _region', 'clustered']
group_file = ['result/p1/category_category1_clustered_3.csv', 'result/p1/category_category2_clustered_4.csv',
'result/p1/category_category3_clustered_4.csv',
                'result/p1/category_seller_category_clustered_4.csv',
'result/p1/category_inventory_category_clustered_4.csv',
                'result/p1/category_seller_level_clustered_4.csv', 'result/p1/category_warehouse
_category_clustered_2.csv',
                'result/p1/category_warehouse _region_clustered_3.csv','result/p1/category_clustered_5.csv']
group_index = 8

# 1. 数据加载和预处理
clusters = pd.read_csv(group_file[group_index])
raw_data = pd.read_csv('raw/1.csv')
raw_data['date'] = pd.to_datetime(raw_data['date'])
raw_data = raw_data.sort_values(by='date')

# 2. 按照 Cluster 分类
groups = clusters.groupby('Cluster')

def extract_features(series, last_date):
    """
    提取给定窗口数据的特征
    :param window: 大小为 WINDOWS_SIZE 的时间窗口数据
    :return: list, 提取的特征
    """
    # 滞后特征
    # 倒序 1 天到倒序 7 天的需求量之和
    lag_features = [np.sum(series[-i:]) for i in range(1, 8)]
    # 倒序 7 天、14 天、21 天、28 天的需求量均值、标准差
    rolling_mean_features =
[np.mean(series[-7:]),np.mean(series[-14:]),np.mean(series[-21:]),np.mean(series[-28:])]
    rolling_std_features = [np.std(series[-7:]),np.std(series[-14:]),np.std(series[-21:]),np.std(series[-28:])]
    # 数据的最大值和最小值
    max_val = np.max(series)
    min_val = np.min(series)
    # 这 WINDOWS_SIZE 天数据的趋势性强度
    trend_strength = np.polyfit(range(WINDOWS_SIZE), series, 1)[0] # 使用线性拟合的斜率作为趋势性强
度
    # 计算周期性/季节性趋势强度
    fft_values = np.fft.fft(series)
    magnitude = np.abs(fft_values[:len(series) // 2])
    dominant_frequency_magnitude = np.max(magnitude[1:])  # 排除直流分量
    # 从 last_date 获取'day'和'weekday'
    # last_date = dates.iloc[-1]
    day = last_date.day
    weekday = last_date.weekday() # 0 代表星期一，6 代表星期天
    # 将所有特征合并到一个列表中并返回
    return lag_features + rolling_mean_features + rolling_std_features + [max_val, min_val, trend_strength,
dominant_frequency_magnitude, day, weekday]

# 函数：计算 1-WMAPE
def compute_wmape(y_true, y_pred):
    assert len(y_true) == len(y_pred), "y_true and y_pred must have the same length"
    return 1 - np.sum(np.abs(y_true - y_pred)) / np.sum(y_true) if np.sum(y_true) != 0 else 1 -
np.count_nonzero(y_pred)/len(y_true)

def compute_rmsse(y_true, y_pred, y_train):
```

```
    # Find the index of the first non-zero demand
    start_index = np.argmax(y_train > 0)
    # If no non-zero demands are found, start from the beginning
    if start_index == 0 and y_train[start_index] == 0:
        start_index = 0
    # Adjust y_train to consider only periods after the first non-zero demand
    adjusted_y_train = y_train[start_index:]
    numerator = np.sum((y_true - y_pred) ** 2)
    denominator = np.sum((adjusted_y_train[1:] - adjusted_y_train[:-1]) ** 2) / (len(adjusted_y_train) - 1)
    if denominator == 0:
        return np.inf
    return np.sqrt(numerator / denominator)

# 函数：计算 MAPE
def compute_mape(y_true, y_pred):
    y_true_adjusted = np.where(y_true == 0, 0.1, y_true)  # 使用 np.where 将 y_true 中的 0 替换为 0.1
    return np.mean(np.abs((y_true - y_pred) / y_true_adjusted)) * 100

# 函数：计算 RMSE
def compute_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

all_predictions = []
all_actuals = []
all_predictions_dfs = []
# 列表保存每个组合的预测结果的评价指标
all_wmapes, all_rmsses, all_mapes, all_rmses, all_r2s = [], [], [], [], []
# 列表保存每一个大类总需求量预测的评价指标
cluster_wmapes, cluster_rmsses, cluster_mapes, cluster_rmses, cluster_r2s = [], [], [], [], []
for cluster_id, group in groups:
    print(cluster_id)
    print(group)
    cluster_data = pd.DataFrame()
    for _, row in group.iterrows():
        combo_data = raw_data[(raw_data['seller_no'] == row['seller_no']) &
                              (raw_data['product_no'] == row['product_no']) &
                              (raw_data['warehouse_no'] == row['warehouse_no']) &
                              (raw_data['date'] >= '2022-12-01') & (raw_data['date'] <= '2023-05-15')]
        combo_data = combo_data.sort_values(by='date')
        cluster_data = pd.concat([cluster_data, combo_data])
    # 总需求量时间序列
    total_demand_series = cluster_data.groupby('date')['qty'].sum()
    # print(total_demand_series)

    X, Y = [], []
    for i in range(len(total_demand_series) - WINDOWS_SIZE):
        features = extract_features(total_demand_series.iloc[i:i+WINDOWS_SIZE].values,
cluster_data['date'].iloc[i+WINDOWS_SIZE-1])
        X.append(features)
        Y.append(total_demand_series.iloc[i+WINDOWS_SIZE])
    # 取最后 15 天，即 2023/5/1 到 2023/5/15 的数据作为验证集
    X_train, X_val = np.array(X[:-15]), np.array(X[-15:])
    Y_train, Y_val = np.array(Y[:-15]), np.array(Y[-15:])

    dataset_train = lgb.Dataset(X_train, label=Y_train)
    dataset_validation = lgb.Dataset(X_val, label=Y_val)
    params = {
        'boosting_type': 'gbdt',
        'objective': 'tweedie',
```

```
            'metric': 'rmse',
            'learning_rate': 0.01,
        }
        # 设置回调函数
        callbacks = [lgb.log_evaluation(period=1), lgb.early_stopping(stopping_rounds=10)]
        model = lgb.train(params, dataset_train, num_boost_round=500, valid_sets=dataset_validation,
callbacks=callbacks)
        # 保存大类预测模型
        joblib.dump(model,
'saved_models/model_category_{}_cluster_{}.pkl'.format(group_list[group_index],cluster_id))

        # 总需求量验证集预测，计算评价指标，保存
        total_demand_pred = model.predict(X_val)
        cluster_wmapes.append(compute_wmape(Y_val, total_demand_pred))
        cluster_rmsses.append(compute_rmsse(Y_val, total_demand_pred, Y_train))
        cluster_mapes.append(compute_mape(Y_val, total_demand_pred))
        cluster_rmses.append(compute_rmse(Y_val, total_demand_pred))
        cluster_r2s.append(r2_score(Y_val, total_demand_pred))

        # 逐日预测 2023/5/16 到 2023/5/30
        future_predictions = []
        last_window = total_demand_series.iloc[-WINDOWS_SIZE:].values
        last_dates = total_demand_series.index[-WINDOWS_SIZE:].to_list()
        for i in range(15):
            features = extract_features(last_window, last_dates[-1])
            X_future = np.array([features])
            # X_future = scaler.fit_transform(X_future)
            future_pred = model.predict(X_future)
            future_predictions.append(future_pred[0])
            new_date = last_dates[-1] + pd.Timedelta(days=1)
            last_dates.append(new_date)
            last_window = np.append(last_window[1:], future_pred)

        # 计算每类中各组合的预测结果
        for _, row in group.iterrows():
            combo_data = raw_data[(raw_data['seller_no'] == row['seller_no']) &
                                  (raw_data['product_no'] == row['product_no']) &
                                  (raw_data['warehouse_no'] == row['warehouse_no']) &
                                  (raw_data['date'] >= '2022-12-01') & (raw_data['date'] <= '2023-05-15')]
            combo_data = combo_data.sort_values(by='date')
            # 计算权重，取 2023/4/1 到 2023/4/30 作为验证集权重的计算
            last_15_days = combo_data.iloc[-15:]
            last_days = combo_data.iloc[-WINDOWS_SIZE-15:-15]
            weights = last_days['qty'] / total_demand_series.iloc[-15-WINDOWS_SIZE:-15].values
            avg_weight = weights.mean()
            # print(weights, avg_weight)

            # 计算预测值
            combo_pred_float = total_demand_pred * avg_weight
            combo_pred = np.round(total_demand_pred * avg_weight)  # 使用 np.round 四舍五入

            # 计算当前组合的评估指标
            wmape = compute_wmape(last_15_days['qty'].values, combo_pred)
            rmsse = compute_rmsse(last_15_days['qty'].values, combo_pred, np.array(combo_data['qty']))
            # mape = compute_mape(last_15_days['qty'].values, combo_pred)
            rmse = compute_rmse(last_15_days['qty'].values, combo_pred)
            r2 = r2_score(last_15_days['qty'].values, combo_pred)
            all_wmapes.append(wmape)
            all_rmsses.append(rmsse)
```

```
            # all_mapes.append(mape)
            all_rmses.append(rmse)
            all_r2s.append(r2)

            df_pred = pd.DataFrame({
                'seller_no': row['seller_no'],
                'product_no': row['product_no'],
                'warehouse_no': row['warehouse_no'],
                'date': pd.date_range(start="2023-05-1", end="2023-05-15"),
                'predicted_value_float': combo_pred_float,
                'predicted_value': combo_pred,
                '1-WMAPE': [wmape] * 15,
                'RMSSE': [rmsse] * 15,
                # 'MAPE': [mape] * 15,
                'RMSE': [rmse] * 15,
                'R2': [r2] * 15
            })
            all_predictions_dfs.append(df_pred)  # 将当前预测结果加入到汇总列表中
            all_predictions.extend(combo_pred)
            all_actuals.extend(last_15_days['qty'].values)

            # 计算权重，取 2023/4/16 到 2023/5/15 作为未来 15 天预测值权重的计算
            last_days = combo_data.iloc[-WINDOWS_SIZE:]
            weights = last_days['qty'] / total_demand_series.iloc[-WINDOWS_SIZE:].values
            avg_weight = weights.mean()
            # 使用上面计算的 future_predictions 计算预测值
            combo_pred_future_float = np.array(future_predictions) * avg_weight
            combo_pred_future = np.round(np.array(future_predictions) * avg_weight)

            df_pred_future = pd.DataFrame({
                'seller_no': row['seller_no'],
                'product_no': row['product_no'],
                'warehouse_no': row['warehouse_no'],
                'date': pd.date_range(start="2023-05-16", end="2023-05-30"),
                'predicted_value_float': combo_pred_future_float,
                'predicted_value': combo_pred_future
            })
            all_predictions_dfs.append(df_pred_future)  # 将未来的预测结果加入到汇总列表中

# 合并所有的预测结果
final_predictions = pd.concat(all_predictions_dfs, ignore_index=True)

# 保存到 CSV 文件
final_predictions.to_csv('result/p1/forecast_lightgbm/forecast_lightgbm_{}.csv'.format(group_list[group_index]),
index=False)

# 评估模型
print(f"Average 1-WMAPE: {np.mean(all_wmapes)}")
print(f"Average RMSSE: {np.mean(all_rmsses)}")
# print(f"Average MAPE: {np.mean(all_mapes)}")
print(f"Average RMSE: {np.mean(all_rmses)}")
print(f"Average R2: {np.mean(all_r2s)}")
print("total_demand_pred 1-WMAPE:", cluster_wmapes)
print("total_demand_pred RMSSE:", cluster_rmsses)
print("total_demand_pred MAPE:", cluster_mapes)
print("total_demand_pred RMSE:", cluster_rmses)
print("total_demand_pred R2:", cluster_r2s)
```

No: 5	Filename: p2-cluster-Kmeans.ipynb

```
import pandas as pd
import numpy as np
from scipy.stats import linregress, skew, kurtosis
from tsfresh import extract_features
from sklearn.cluster import KMeans
from sklearn.preprocessing import OneHotEncoder
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import acf

# 读入数据
df1 = pd.read_csv("raw/1.csv")
df2 = pd.read_csv("raw/2.csv")
df3 = pd.read_csv("raw/3.csv")
df4 = pd.read_csv("raw/4.csv")
df5 = pd.read_csv("raw/5.csv")

# 合并数据
df = pd.concat([df1, df5], ignore_index=True)
df = df.merge(df2, on='product_no', how='left')
df = df.merge(df3, on='seller_no', how='left')
df = df.merge(df4, on='warehouse_no', how='left')
df['date'] = pd.to_datetime(df['date'])
print(df)

# 提取指定时间段的特征
time_ranges = [('2023-04-25', '2023-05-01'), ('2023-05-02', '2023-05-08'), ('2023-05-09', '2023-05-15'),
('2023-04-23', '2023-05-10')]

results = []
group = df.groupby(['seller_no', 'product_no', 'warehouse_no'])
for (seller, product, warehouse), sub_data in group:
    sub_data = sub_data.sort_values(by='date')  # 确保日期是升序排列的
    series_data = []
    for start_date, end_date in time_ranges:
        data_range = sub_data[(sub_data['date'] >= start_date) & (sub_data['date'] <= end_date)]['qty']
        mean = data_range.mean()
        std_val = data_range.std()
        max_val = data_range.max()
        min_val = data_range.min()
        trend_strength = linregress(range(len(data_range)), data_range.values).slope
        if std_val == 0:
            skewness_val = 0
            kurtosis_val = 0
        else:
            skewness_val = data_range.skew()
            kurtosis_val = data_range.kurtosis()
        series_data.extend([mean, std_val, max_val, min_val, trend_strength, skewness_val, kurtosis_val])
    data_range = sub_data[(sub_data['date'] >= '2023-04-23') & (sub_data['date'] <= '2023-05-10')]['qty']
    # 傅里叶变换，寻找主频分量
    data_fft = np.fft.fft(data_range)
    n = len(data_range)
    # 计算频率和功率谱密度（PSD）
    freqs = np.fft.fftfreq(n, 1/7)
    psd = np.abs(data_fft) ** 2# 提取功率谱：频率谱是傅里叶变换的结果，它表示信号在不同频率上的
能量分布。我们可以通过提取频率谱来分析数据中的周期性波动。
    # 忽略负频率
    pos_mask = freqs > 0
    freqs = freqs[pos_mask]
```

```
        psd = psd[pos_mask]
        # 找到主频率及其幅度
        peak_freq = freqs[np.argmax(psd[1:])]
        peak_amplitude = np.max(psd)
        series_data.extend([peak_freq])
        results.append([seller, product, warehouse] + series_data)

feature_names = ['mean', 'std_dev', 'max', 'min', 'trend_strength', 'skewness', 'kurtosis']
columns = ['seller_no', 'product_no', 'warehouse_no']
# 更新列名以包括 One-hot 编码的列
for i in range(len(time_ranges)):
        for feature in feature_names:
                columns.append(f"{feature}_{i+1}")
# columns.append("trend_strength")
# columns.append("seasonal_strength")
columns.append("peak_freq")
result_df = pd.DataFrame(results, columns=columns)

# 保存特征到 CSV 文件
result_df.to_csv('result/p2/cluster-features-mathematical.csv', index=False)
# 将分类特征转为编码
categorical_cols = ['category1', 'category2', 'category3', 'seller_category', 'inventory_category', 'seller_level',
'warehouse _category', 'warehouse _region']
df_onehot = pd.get_dummies(df, columns=categorical_cols)

# 去除'date'和'qty'列
df_onehot = df_onehot.drop(columns=['date', 'qty'])

# 按照商家-商品-仓库组合进行分组，并合并重复条目
grouped_df = df_onehot.groupby(['seller_no', 'product_no', 'warehouse_no']).first().reset_index()

import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# 使用 KMeans 进行聚类分析前，先标准化数据
scaler = StandardScaler()
features = grouped_df.drop(columns=['seller_no', 'product_no', 'warehouse_no'])
scaled_features = scaler.fit_transform(features)

# 使用肘部法则
#"""
# 法 1：肘部法则 (Elbow Method)：该方法涉及运行 k-means 聚类在数据集上多次，每次使用不同的 k
值。每次执行算法都会得到一个与之前的 k 值不同的 SSE（每个点到其聚类中心的平方距离之和）。
# 最终，SSE 值会被绘制出来，我们选择“肘部”处的 k 值。
# 法 2：轮廓系数 (Silhouette Coefficient)：轮廓系数是一个用于评估聚类效果好坏的指标，其值介于-1
和 1 之间。接近 1 的值表示样本与其所在的聚类非常相似，接近-1 的值表示样本与其邻近的聚类更为相
似。
wcss = [] # Within-cluster sum of squares
sil_coefficients = []
possible_k_values = range(2, 21)  # 例如测试 2 到 10 的 k 值

for k in possible_k_values:
        kmeans = KMeans(n_clusters=k, random_state=42).fit(scaled_features)
        wcss.append(kmeans.inertia_)
        sil_coefficients.append(silhouette_score(scaled_features, kmeans.labels_, metric='euclidean'))
```

```
# 肘部法则的可视化
plt.figure()
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.plot(possible_k_values, wcss, '-o')
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.title('肘部法则确定最优 k 值',fontsize=16)
plt.xlabel('k',fontsize=14)
plt.ylabel('簇内平方和',fontsize=14)
plt.show()

# 轮廓系数的可视化
plt.figure()
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.plot(possible_k_values, sil_coefficients, '-o')
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.title('轮廓系数确定最优 k 值',fontsize=16)
plt.xlabel('k',fontsize=14)
plt.ylabel('轮廓系数',fontsize=14)
plt.show()

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 读取两个文件
df_mathematical = pd.read_csv("result/p2/cluster-features-mathematical.csv")
df_variable = pd.read_csv("result/p2/cluster-features-variable.csv")

# 按'seller_no', 'product_no', 'warehouse_no'合并数据
merged_df = pd.merge(df_mathematical, df_variable, on=['seller_no', 'product_no', 'warehouse_no'])

# 删除'seller_no', 'product_no', 'warehouse_no'列，留下特征列
features_df = merged_df.drop(['seller_no', 'product_no', 'warehouse_no'], axis=1)

# 标准化数据
scaler = StandardScaler()
scaled_data = scaler.fit_transform(features_df)

wcss = [] # Within-cluster sum of squares
sil_coefficients = []
possible_k_values = range(2, 21)    # 例如测试 2 到 10 的 k 值

for k in possible_k_values:
    kmeans = KMeans(n_clusters=k, random_state=42).fit(scaled_data)
    wcss.append(kmeans.inertia_)
    sil_coefficients.append(silhouette_score(scaled_data, kmeans.labels_, metric='euclidean'))

# 肘部法则的可视化
plt.figure()
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.plot(possible_k_values, wcss, '-o')
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
```

```
plt.title('肘部法则确定最优 k 值',fontsize=16)
plt.xlabel('k',fontsize=14)
plt.ylabel('簇内平方和',fontsize=14)
plt.show()

# 轮廓系数的可视化
plt.figure()
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.plot(possible_k_values, sil_coefficients, '-o')
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.title('轮廓系数确定最优 k 值',fontsize=16)
plt.xlabel('k',fontsize=14)
plt.ylabel('轮廓系数',fontsize=14)
plt.show()

# 为了更好地理解聚类的效果，可以使用 PCA 进行降维并可视化聚类结果
best_k = 14  # 根据之前的分析确定的最佳 k 值，假设最佳 k 值为 5，你可以根据上一个分析来替换此值
kmeans = KMeans(n_clusters=best_k, random_state=42).fit(scaled_data)
merged_df['Cluster'] = kmeans.labels_ # 将聚类标签添加到原始数据中
merged_df.to_csv("result/p2/clustered_{}.csv".format(best_k), index=False) # 保存带有聚类标签的新数据

# 降维到 2D
pca_2d = PCA(n_components=2)
principal_components_2d = pca_2d.fit_transform(scaled_features)

# 可视化 2D
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.figure(figsize=(8, 6))
plt.scatter(principal_components_2d[:, 0], principal_components_2d[:, 1], c=kmeans.labels_, cmap='rainbow')
plt.title('2D PCA')
plt.xlabel('主成分 1')
plt.ylabel('主成分 2')
plt.colorbar()
plt.grid(True)
plt.show()

# 降维到 3D
pca_3d = PCA(n_components=3)
principal_components_3d = pca_3d.fit_transform(scaled_features)

# 可视化 3D
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(principal_components_3d[:, 0], principal_components_3d[:, 1], principal_components_3d[:, 2], c=kmeans.labels_, cmap='rainbow')
ax.set_title('3D PCA')
ax.set_xlabel('主成分 1')
ax.set_ylabel('主成分 2')
ax.set_zlabel('主成分 3')
plt.show()
```

No: 6	Filename: p2-forecast-knn.ipynb

```
import pandas as pd
import numpy as np
import pickle
import joblib
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

WINDOWS_SIZE = 14

# 数据加载和预处理
clusters = pd.read_csv("result/p2/clustered_14.csv")
df1 = pd.read_csv("raw/1.csv")
df5 = pd.read_csv("raw/5.csv")
df = pd.concat([df1, df5], ignore_index=True)
df['date'] = pd.to_datetime(df['date'])
df5['date'] = pd.to_datetime(df5['date'])
df = df.sort_values(by='date')
df5 = df5.sort_values(by='date')

# 函数：计算 1-WMAPE
def compute_wmape(y_true, y_pred):
    assert len(y_true) == len(y_pred), "y_true and y_pred must have the same length"
    return 1 - np.sum(np.abs(y_true - y_pred)) / np.sum(y_true) if np.sum(y_true) != 0 else 1 - np.count_nonzero(y_pred)/len(y_true)

def compute_rmsse(y_true, y_pred, y_train):
    # Find the index of the first non-zero demand
    start_index = np.argmax(y_train > 0)
    # If no non-zero demands are found, start from the beginning
    if start_index == 0 and y_train[start_index] == 0:
        start_index = 0
    # Adjust y_train to consider only periods after the first non-zero demand
    adjusted_y_train = y_train[start_index:]
    numerator = np.sum((y_true - y_pred) ** 2)
    denominator = np.sum((adjusted_y_train[1:] - adjusted_y_train[:-1]) ** 2) / (len(adjusted_y_train) - 1)
    if denominator == 0:
        return np.inf
    return np.sqrt(numerator / denominator)

# 函数：计算 MAPE
def compute_mape(y_true, y_pred):
    y_true_adjusted = np.where(y_true == 0, 0.1, y_true)  # 使用 np.where 将 y_true 中的 0 替换为 0.1
    return np.mean(np.abs((y_true - y_pred) / y_true_adjusted)) * 100

# 函数：计算 RMSE
def compute_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

# 按照 Cluster 分类
groups = clusters.groupby('Cluster')

# 存储预测结果
all_predictions = []
all_scores = []
best_params_list = []
```

```
for cluster_id, group in groups:
    features, labels = [], []
    # 滑动窗口提取特征
    for _, row in group.iterrows():
        combo_data = df[(df['seller_no'] == row['seller_no']) &
                        (df['product_no'] == row['product_no']) &
                        (df['warehouse_no'] == row['warehouse_no']) &
                        (df['date'] >= '2022-12-01') & (df['date'] <= '2023-05-15')]
        combo_data = combo_data.sort_values(by='date')

        for i in range(len(combo_data) - WINDOWS_SIZE):
            feature_i = combo_data['qty'].iloc[i:i+WINDOWS_SIZE].values
            features.append(feature_i)
            labels.append(combo_data['qty'].iloc[i+WINDOWS_SIZE])

    X = np.array(features)
    y = np.array(labels)

    tscv = TimeSeriesSplit(n_splits=5)
    models, scores, model_params = [], [], []
    for train_index, test_index in tscv.split(X):
        X_train, X_test = X[train_index], X[test_index]
        y_train, y_test = y[train_index], y[test_index]

        # 标准化特征
        scaler = StandardScaler()
        X_train = scaler.fit_transform(X_train)
        X_test = scaler.transform(X_test)

        # 定义参数网格
        param_grid = {
            'n_neighbors': [3, 5, 7, 10],
            'weights': ['uniform', 'distance'],
            'metric': ['euclidean']
        }

        # 使用 GridSearchCV 进行参数寻优
        knn = KNeighborsRegressor()
        grid_search = GridSearchCV(knn, param_grid, cv=3, scoring='neg_mean_squared_error', verbose=1,
n_jobs=-1)
        grid_search.fit(X_train, y_train)

        # 获取最佳模型
        model = grid_search.best_estimator_
        best_params = grid_search.best_params_
        print(best_params)

        # model = SVR()
        # model.fit(X_train, y_train)

        y_pred = model.predict(X_test)

        # 计算指标
        wmape = compute_wmape(y_test, y_pred)
        rmsse = compute_rmsse(y_test, y_pred, y_train)
        mape = compute_mape(y_test, y_pred)
        rmse = compute_rmse(y_test, y_pred)
        models.append(model)
        scores.append([wmape, rmsse, mape, rmse])
```

```
            model_params.append(best_params)

        for i in range(len(scores)):
            print(f"1-WMAPE: {scores[i][0]}, RMSSE: {scores[i][1]}, MAPE: {scores[i][2]}, RMSE: {scores[i][3]}")
        all_scores.append([np.mean(scores, axis=0).tolist()])
        best_params_list.append(model_params)
        for i in range(len(models)):
            joblib.dump(models[i], 'saved_models/p2/knn/model_cluster_{}_fold_{}.pkl'.format(cluster_id, i))

        for _, row in group.iterrows():
            combo_data = df5[(df5['seller_no'] == row['seller_no']) &
                             (df5['product_no'] == row['product_no']) &
                             (df5['warehouse_no'] == row['warehouse_no'])]
            combo_data = combo_data.sort_values(by='date')
            if combo_data.empty:
                continue
            else:
                # 选取最后 WINDOWS_SIZE 大小的数据
                last_window = combo_data.iloc[-WINDOWS_SIZE:]['qty'].values
                predictions = []
                predictions_int = []
                for _ in range(15):  # 预测 15 天
                    feature = last_window[-WINDOWS_SIZE:]
                    # 使用每个模型预测
                    model_predictions = [model.predict(scaler.transform([feature])) for model in models]
                    # 计算五个模型的平均预测值
                    avg_prediction = np.mean(model_predictions)
                    predictions.append(avg_prediction)
                    predictions_int.append(np.round(avg_prediction))
                    # 更新窗口数据，将预测值加入窗口并移除窗口的第一个值
                    last_window = np.append(last_window[1:], avg_prediction)

                df_predictions = pd.DataFrame({
                    'seller_no': row['seller_no'],
                    'product_no': row['product_no'],
                    'warehouse_no': row['warehouse_no'],
                    'date': pd.date_range(start="2023-05-16", end="2023-05-30"),
                    'predictions': predictions,
                    'predictions_int': predictions_int
                })
                all_predictions.append(df_predictions)

print(best_params_list)
print(all_scores)
for i in range(len(all_scores)):
    print(all_scores[i])

# 合并所有的预测结果
final_predictions = pd.concat(all_predictions, ignore_index=True)

# 保存到 CSV 文件
final_predictions.to_csv('result/p2/forecast_knn/forecast_knn.csv', index=False)
```

No: 7	Filename: p3-calculate-weights.ipynb

```
import pandas as pd
import numpy as np
import lightgbm as lgb
```

```
import pickle
import joblib
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler

# 1. 数据加载和预处理
raw_data = pd.read_csv('raw/1.csv')
raw_data['date'] = pd.to_datetime(raw_data['date'])
raw_data = raw_data.sort_values(by='date')

november_data = pd.read_csv('raw/6.csv')
november_data['date'] = pd.to_datetime(november_data['date'])
november_data = november_data.sort_values(by='date')

groups = raw_data.groupby(['seller_no', 'product_no', 'warehouse_no'])

factor_results = []
for group_id, group in groups:
    combo_data = group.sort_values(by='date')
    # print(combo_data)

    date_ranges = [
        ('2022-12-01', '2022-12-11'),
        ('2023-01-01', '2023-01-11'),
        ('2023-02-01', '2023-02-11'),
        ('2023-03-01', '2023-03-11'),
        ('2023-04-01', '2023-04-11'),
        ('2023-05-01', '2023-05-11')
    ]
    daily_averages = [0] * 11  # 初始化一个长度为 11 的列表
    for start, end in date_ranges:
        subset = combo_data[(combo_data['date'] >= start) & (combo_data['date'] <= end)]
        # print(subset)
        for i in range(11):
            daily_value = subset[subset['date'] == pd.to_datetime(start) + pd.DateOffset(days=i)]['qty'].values[0]
            # print(i, daily_value)
            daily_averages[i] += daily_value
    # 对每一天的均值求平均
    daily_averages = [x / 6 for x in daily_averages]
    # print(daily_averages)

    pomo_ranges = ('2022-11-01', '2022-11-11')

    # 获取 11 月促销期的需求量向量
    combo_promo_data = november_data[
        (november_data['seller_no'] == combo_data.iloc[0]['seller_no']) &
        (november_data['product_no'] == combo_data.iloc[0]['product_no']) &
        (november_data['warehouse_no'] == combo_data.iloc[0]['warehouse_no'])
    ]
    # print(combo_promo_data)
    if combo_promo_data.empty:
        for i in range(11):
            date = pd.to_datetime('2022-11-01') + pd.DateOffset(days=i)
            new_row = {'seller_no': combo_data.iloc[0]['seller_no'], 'product_no': combo_data.iloc[0]['product_no'],
                       'warehouse_no': combo_data.iloc[0]['warehouse_no'], 'date': date, 'qty': daily_averages[i]}
```

```
                # combo_promo_data = combo_promo_data.append(new_row, ignore_index=True)
                combo_promo_data = pd.concat([combo_promo_data, pd.DataFrame([new_row])], ignore_index=True)
        else:
            for i in range(11):
                date = pd.to_datetime('2022-11-01') + pd.DateOffset(days=i)
                if date not in combo_promo_data['date'].values:
                    # 使用日常均值填充缺失值
                    qty_value = daily_averages[i]
                    new_row = {'seller_no': combo_data.iloc[0]['seller_no'], 'product_no': combo_data.iloc[0]['product_no'],
                               'warehouse_no': combo_data.iloc[0]['warehouse_no'], 'date': date, 'qty': qty_value}
                    # combo_promo_data = combo_promo_data.append(new_row, ignore_index=True)
                    combo_promo_data = pd.concat([combo_promo_data, pd.DataFrame([new_row])], ignore_index=True)
        # 对 combo_promo_data 按日期排序
        combo_promo_data = combo_promo_data.sort_values(by='date')
        promo_vector = combo_promo_data['qty'].values.tolist()
        adjustment_factors = []
        for i in range(len(daily_averages)):
            if promo_vector[i]==0:
                adjustment_factors.append(1)
            else:
                if daily_averages[i]==0:
                    adjustment_factors.append(1)
                else:
                    adjustment_factors.append(promo_vector[i]/daily_averages[i])
        # adjustment_factors = promo_vector/np.array(daily_averages)
        factor1 = np.mean(adjustment_factors[:5])
        factor2 = np.mean(adjustment_factors[5:10])
        factor3 = adjustment_factors[10]
        factor1 = factor1 if factor1<10 else 10
        factor2 = factor2 if factor2<10 else 10
        factor3 = factor3 if factor3<10 else 10
        factor_result = {'seller_no': combo_data.iloc[0]['seller_no'], 'product_no': combo_data.iloc[0]['product_no'],
                         'warehouse_no': combo_data.iloc[0]['warehouse_no'], 'factor1': factor1, 'factor2': factor2,'factor3': factor3}
        # 将字典转化为 DataFrame 并添加一个索引
        df_factor = pd.DataFrame([factor_result])
        factor_results.append(df_factor)

df_factor_result = pd.concat(factor_results, ignore_index=True)
df_factor_result.to_csv('result/p3/calculate_weights_new.csv', index=False)

group_list = ['category1', 'category2', 'category3', 'seller_category', 'inventory_category', 'seller_level', 'warehouse _category', 'warehouse _region', 'clustered']
forecast_file = ['result/p3/forecast_lightgbm_category1.csv', 'result/p3/forecast_lightgbm_category2.csv', 'result/p3/forecast_lightgbm_category3.csv',
                 'result/p3/forecast_lightgbm_seller_category.csv', 'result/p3/forecast_lightgbm_inventory_category.csv', 'result/p3/forecast_lightgbm_seller_level.csv',
                 'result/p3/forecast_lightgbm_warehouse _category.csv', 'result/p3/forecast_lightgbm_warehouse _region.csv', 'result/p3/forecast_lightgbm_clustered.csv']
group_index = 8
# 加载数据
forecast_df = pd.read_csv(forecast_file[group_index])
weights_df = pd.read_csv('result/p3/calculate_weights_new.csv')
```

```
# 将'date'列转化为日期格式
forecast_df['date'] = pd.to_datetime(forecast_df['date'])

# 提取 2023/6/1 到 2023/6/20 的数据
forecast_df = forecast_df[(forecast_df['date'] >= '2023-06-01') & (forecast_df['date'] <= '2023-06-20')]

def apply_weights(row):
    if row['date'] <= pd.Timestamp('2023-06-09'):
        return row['predicted_value_float'] * row['factor1']
    elif row['date'] <= pd.Timestamp('2023-06-17'):
        return row['predicted_value_float'] * row['factor2']
    else:
        return row['predicted_value_float'] * row['factor3']

# 合并 forecast_df 和 weights_df
merged_df = forecast_df.merge(weights_df, on=['seller_no', 'product_no', 'warehouse_no'], how='left')

# 计算加权的预测值
merged_df['weighted_prediction'] = merged_df.apply(apply_weights, axis=1)

# 保存结果到 new.csv
result_df = merged_df[['seller_no', 'product_no', 'warehouse_no', 'date', 'weighted_prediction']]
result_df.to_csv('result/p3/forecast_adjust_{}.csv'.format(group_list[group_index]), index=False)
```